

VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI
School of Electronics Engineering
Department of Electronics and Communication Engineering

Runtime Parameter Adaptation for Linux Queue Disciplines

A Scaling Law in `target`/RTT

Technical Report
Complete Methodology, Results and Comparison

Submitted by

Amritha S | Yugeshwaran P | Deepti Annuncia

Every measurement in this report was produced on a purpose-built Linux guest and is reproducible from the committed logs by the commands in Appendix D.

October 5, 2026

Abstract

Linux ships `fq_codel` as its default queue discipline with four parameters fixed at configuration time. Whether adapting them at runtime is worth doing has been answered both ways in the literature. This report establishes that both answers are correct and that a single dimensionless number decides which applies. Across 178 measured runs spanning three workloads, eight alternative queue disciplines and path round-trip times from 2 to 200 ms, the benefit of adaptation is governed not by path RTT but by the ratio $r = \text{target}/\text{RTT}$, saturating at 20.5% with half of that reached at $r = 0.50$. Published guidance places a correctly configured deployment at r between 0.05 and 0.10, so half the available benefit arrives five to ten times outside that envelope: adaptation is a remedy for misconfiguration rather than an improvement on correct configuration. The report gives the full methodology, the complete result set with every figure described at its point of use, a parameter-level comparison against the base paper and six recent systems, and the nine measurement defects found and corrected during the work.

Keywords: active queue management; bufferbloat; `fq_codel`; CoDel; runtime parameter adaptation; dimensionless scaling law; Linux traffic control; eBPF telemetry; pre-registered evaluation.

Contents

Abstract	1
1 Introduction	8
2 Literature survey	8
2.1 The problem and the standard it set	8
2.2 Drop-probability schemes	8
2.3 Delay-target schemes	8
2.4 Runtime adaptation, the line this work sits in	9
2.5 What that line does not supply	9
2.6 Programmable telemetry	9
3 Methodology	9
3.1 Why a purpose-built kernel was required	9
3.2 Topology, and why the first one was wrong	10
3.3 Flow telemetry, and the silent failure it had	12
3.4 Controller	12
3.5 Measurement	14
3.6 Experimental design	14
4 Results	15
4.1 Step 1: flow queueing dominates everything else	15
4.2 Step 2: the two latencies are not interchangeable	17
4.3 Step 3: separating the controller's cost from its decisions	17
4.4 Step 4: what adaptation achieves, parameter by parameter	20
4.5 Step 5: under a load that changes, adaptation does not help	20
4.6 Step 6: an alternative requiring no controller does better	23
4.7 Step 7: the predictive path never engaged	23
4.8 Step 8: sparse flows under mixed traffic	23
5 Comparison	24
5.1 Disciplines under test and their parameters	24
5.2 Against the base paper and six recent systems	24
5.3 What none of the prior systems reports	25
6 The scaling law	25
6.1 From an unexplained table to a hypothesis	25
6.2 Pre-registration	25
6.3 Result	25
6.4 What it costs, and what it means	26

7	Measurement defects found and corrected	27
8	Threats to validity	27
9	Conclusion	28
A	Complete figure set	28
B	Implementation captures not shown above	28
C	Per-run figures	28
D	Reproducing every number	44

List of Figures

1	Session kernel cannot run the AQMs under study. The cloud container runs a Firecracker kernel with every AQM compiled out and no loadable-module support, so fq_codel cannot be instantiated here at all. This is why the experiments run inside a purpose-built VM rather than directly.	10
2	Kernel built with every AQM compiled in. A 6.12.48 kernel was configured and built with fq_codel, CoDel, CAKE, PIE, FQ-PIE, RED, netem, TBF and HTB all built in (=y), plus BTF for eBPF. This kernel is what the experiments run on. . .	10
3	AQM availability confirmed inside the VM. Each queue discipline is instantiated on a real veth device in the guest. Availability is demonstrated, not assumed. . . .	11
4	The measurement topology as configured. A client namespace offers the load, a router namespace carries the shaper and the discipline under test, and a server namespace terminates the flows. The shaper sits on the router's egress toward the server, which is the direction bulk data travels, so the queue under study holds data packets.	11
5	The original two-node testbed shaped ACKs, not data. README Parts 2-4 put the bottleneck on veth1 inside ns1 but ran iperf3 FROM ns2, so data flowed ns2 → ns1 while the egress qdisc on veth1 shaped only ns1 → ns2: the acknowledgement path. Left panel: data passes unshaped at multi-Gbit and the queue carries 66-byte ACKs. Right panel: the corrected topology shapes the data path, and the queue carries full-size packets. Every backlog and drop figure in the original Parts 2-4 was a measurement of ACK queueing.	12
6	eBPF telemetry program compiles, attaches and JITs. tc_monitor.c is compiled with BTF, attached at the clsact egress hook on the router, and verified JIT-compiled. Its maps are then located through the bpf(2) syscall.	13
7	eBPF flow telemetry is live under traffic. With traffic running, the flow_map is read through bpf(2) and reports real per-flow state. In all 21,128 samples of the original study this read returned zero.	13
8	Regression tests pin all four defects. Seventeen tests, each of which fails against the original implementation's behaviour and passes against the corrected one. They cover the qdisc-selection bug, the three eBPF decode bugs, the integer-millisecond rounding, and the inert prediction.	15
9	fig01_latency_tail_steady.png. Tail latency (p95 RTT, sparse probe flow) across all systems, log scale (steady workload)	16
10	fig03_throughput_steady.png. Goodput (from iperf3 sum_received), axis zoomed to show it is flat (steady workload)	17
11	fig02_latency_sparse_vs_bulk_steady.png. Sparse probe flow vs bulk TCP flow RTT, why the two differ (steady workload)	18
12	fig04_backlog_steady.png. Mean queue occupancy, log scale (steady workload) . .	18
13	fig06_droprate_steady.png. AQM drop rate and end-to-end TCP retransmissions (steady workload)	19
14	fig13_sham_control_steady.png. Sham-controller condition, separates controller CPU cost from control decisions (steady workload)	19
15	fig12_controller_behaviour.png. What the ACAPE controller actually did: target, backlog, eBPF telemetry, regime	20

16	fig08_allparams_steady.png. All measured parameters $\times$ all systems, normalised heatmap with measured values (steady workload)	21
17	fig07_tradeoff_steady.png. Latency vs throughput trade-off (steady workload) . .	21
18	fig05_fairness_steady.png. Jain's fairness index, axis zoomed (steady workload) . .	22
19	fig10_timeseries_backlog_staged.png. Queue occupancy over time (staged workload)	22
20	fig11_timeseries_rtt_staged.png. Measured RTT over time (20 Hz probe) (staged workload)	23
21	fig14_mixed_workload.png.	24
22	fig15_rtt_sweep.png.	25
23	fig16_scalingLaw.png.	26
24	fig01_latency_tail_staged.png. Tail latency (p95 RTT, sparse probe flow) across all systems, log scale (staged workload)	28
25	fig02_latency_sparse_vs_bulk_staged.png. Sparse probe flow vs bulk TCP flow RTT, why the two differ (staged workload)	29
26	fig03_throughput_staged.png. Goodput (from iperf3 sum_received), axis zoomed to show it is flat (staged workload)	29
27	fig04_backlog_staged.png. Mean queue occupancy, log scale (staged workload) . .	30
28	fig05_fairness_staged.png. Jain's fairness index, axis zoomed (staged workload) . .	30
29	fig06_droprate_staged.png. AQM drop rate and end-to-end TCP retransmissions (staged workload)	31
30	fig07_tradeoff_staged.png. Latency vs throughput trade-off (staged workload) . .	31
31	fig08_allparams_staged.png. All measured parameters $\times$ all systems, normalised heatmap with measured values (staged workload)	32
32	fig09_seeds_backlog_mean_pkts_staged.png. Per-seed values, every individual run shown, mean marked, backlog mean pkts (staged workload)	32
33	fig09_seeds_backlog_mean_pkts_steady.png. Per-seed values, every individual run shown, mean marked, backlog mean pkts (steady workload)	33
34	fig09_seeds_bulk_rtt_mean_ms_staged.png. Per-seed values, every individual run shown, mean marked, bulk rtt mean ms (staged workload)	33
35	fig09_seeds_bulk_rtt_mean_ms_steady.png. Per-seed values, every individual run shown, mean marked, bulk rtt mean ms (steady workload)	34
36	fig09_seeds_jain_staged.png. Per-seed values, every individual run shown, mean marked, jain (staged workload)	34
37	fig09_seeds_jain_steady.png. Per-seed values, every individual run shown, mean marked, jain (steady workload)	35
38	fig09_seeds_retransmits_staged.png. Per-seed values, every individual run shown, mean marked, retransmits (staged workload)	35
39	fig09_seeds_retransmits_steady.png. Per-seed values, every individual run shown, mean marked, retransmits (steady workload)	36
40	fig09_seeds_sparse_rtt_p95_ms_staged.png. Per-seed values, every individual run shown, mean marked, sparse rtt p95 ms (staged workload)	36
41	fig09_seeds_sparse_rtt_p95_ms_steady.png. Per-seed values, every individual run shown, mean marked, sparse rtt p95 ms (steady workload)	37

42	fig09_seeds_throughput_mbps_staged.png. Per-seed values, every individual run shown, mean marked, throughput mbps (staged workload)	37
43	fig09_seeds_throughput_mbps_steady.png. Per-seed values, every individual run shown, mean marked, throughput mbps (steady workload)	38
44	fig10_timeseries_backlog_steady.png. Queue occupancy over time (steady workload)	38
45	fig11_timeseries_rtt_steady.png. Measured RTT over time (20 Hz probe) (steady workload)	39
46	fig13_sham_control_staged.png. Sham-controller condition, separates controller CPU cost from control decisions (staged workload)	39
47	Why the eBPF telemetry read zero in every historical run. Three independent defects in read_flows(), each fatal on its own. The immediate cause was a TypeError swallowed by a bare except; behind it sat wrong struct offsets and a comparison of CLOCK_REALTIME against CLOCK_MONOTONIC giving ages of decades. . .	40
48	Throughput overshoot: sum_sent exceeds the link rate. iperf3's sum_sent counts bytes handed to the socket, not bytes delivered. Under pfifo's 1000-packet buffer the sender fills the queue and reports 12.16 Mbps on a 10 Mbit link - physically impossible as a throughput. The excess appears as an 827-packet backlog and 2381 ms p95 RTT, not as delivered data. Every flow-queueing AQM shows no overshoot. This is why goodput is taken from sum_received.	40
49	Drop rates in the original logs exceed the link's packet rate. A 10 Mbit link carrying 1514-byte packets passes at most 826 packets/s, and iperf3 reported ~179 retransmits/s. The original logs record 100,000-138,000 drops/s - up to 155x the physical packet rate. This is the arithmetic that first showed the measurements could not describe the data path.	41

List of Tables

1	Steady workload: all ten conditions, mean $\pm$ sd across seeds. Eight bulk TCP CUBIC flows with a sparse UDP probe over a 10 Mbit/s bottleneck at 20 ms base RTT.	15
2	Staged workload: the offered load steps during the run, so the queue is never in steady state. This is the workload built to exercise the controller, and the one on which it helps least.	16
3	Scaling-law fit and the pre-registered verdicts. Every threshold in the decision column was committed to the repository before the data existed.	26

1 Introduction

Bufferbloat is the inflation of queueing delay by buffers larger than any control loop requires, while throughput remains unaffected. Linux has converged on `fq_codel` as its response: Deficit Round Robin across roughly a thousand flow queues, with an independent CoDel instance per queue. Four parameters govern it and none changes once set: `target` (5 ms), `interval` (100 ms), `limit` and `quantum`.

Adapting those parameters at runtime has obvious appeal and clear precedent. Adaptive RED adjusts a drop probability by an additive-increase multiplicative-decrease rule; Ye and Leung adapt CoDel’s interval analytically; DESiRED adapts an AQM’s delay target in P4 with deep reinforcement learning. Runtime adaptation of a delay target is therefore not novel and no such claim is made here.

What the literature does not supply is the condition under which adaptation pays. Systems report gains; independent evaluations report none; neither camp states when the other is right. This report establishes that condition experimentally, and it is a dimensionless ratio.

Structure. Section 2 surveys the literature this work sits in and states what it does not supply. Section 3 gives the methodology and the evidence that the apparatus measures what it claims to. Section 4 gives the results as eight steps, each figure placed and described where its argument sits. Section 5 compares this work against the base paper and six recent systems at parameter level. Section 6 derives and tests the scaling law. Section 7 documents the nine measurement defects found during the work. Appendix A holds the complete figure set.

2 Literature survey

2.1 The problem and the standard it set

Gettys and Nichols characterised bufferbloat: buffer memory became cheap faster than queue management improved, so devices acquired buffers larger than any control loop required, and loss-based TCP filled them[1]. RFC 7567 sets the IETF position, recommending that devices deploy AQM by default and explicitly preferring schemes that need no per-flow configuration[2]. That preference for the absence of manual tuning is the standard against which any adaptive scheme has to be judged, and it is why a null result for adaptation is treated here as a reportable outcome rather than a failed experiment.

2.2 Drop-probability schemes

Random Early Detection opened the field, signalling congestion by dropping with a probability derived from average queue length[3]. Its sensitivity to configuration prompted Adaptive RED, which adjusts the maximum drop probability by an additive-increase multiplicative-decrease rule against a queue-length target[4]. PIE replaced queue length with a delay estimate inside a proportional-integral controller[5]. The AIMD policy in the controller evaluated here, including its $\beta = 0.9$ decrease factor, is taken from Adaptive RED, which makes that scheme the closest methodological ancestor of this work.

2.3 Delay-target schemes

CoDel abandoned queue length entirely, dropping when the minimum sojourn time over a sliding interval exceeds a target[6, 7]. Its authors argue that the two parameters are expressed relative to

path round-trip time by construction and therefore require no tuning. The scaling law of Section 6 quantifies the point at which that claim stops holding. `fq_codel` pairs CoDel with Deficit Round Robin across roughly a thousand flow queues and is now the Linux default[8]; CAKE integrates shaping, flow queueing and host fairness[9]; FQ-PIE is the corresponding combination for PIE[10]. L4S standardises a low-latency service class on a dual-queue coupled AQM[11, 12]. All eight of these disciplines are measured here under an identical bottleneck.

2.4 Runtime adaptation, the line this work sits in

Ye and Leung derive stability conditions for CoDel and adapt its `interval` analytically[13]. QueuePilot learns a marking policy for small buffers by reinforcement learning[14], and AQM-LLM distils a language model into a marking controller[15]. DESiRED performs runtime adaptation of an AQM’s delay target in P4 using deep reinforcement learning and in-band telemetry[16]. Runtime adaptation of a delay target is therefore not novel, and no such claim is made here. Toopchinezhad and Ahmadi survey the machine-learning AQM literature and observe that heuristic schemes require careful parameter adjustment, which limits their real-world applicability[17]. SCRR revisits fair-queueing scheduling for modern link rates[18], and Ray et al. characterise how the presence of an AQM distorts speed-test measurement[19].

2.5 What that line does not supply

Every system above reports that its adaptation improved something, and the independent evaluations that find nothing are equally confident. What none of them supplies is the condition under which the other is right. None runs a control condition that separates a controller’s computational cost from the effect of its decisions. None reports sparse-probe and bulk-flow latency separately, despite flow-queueing disciplines privileging sparse flows by construction. None compares against eight alternative disciplines under one identical bottleneck. And none states a numerical prediction before measuring it. The contribution claimed here is accordingly not the adaptation mechanism but the predicate that says when it pays, together with the apparatus required to establish it.

2.6 Programmable telemetry

eBPF at the `tc clsact` hook permits per-flow state to be maintained in kernel maps and read from userspace[20]. The telemetry path used here reads those maps through `bpf(2)` directly rather than by spawning `bpf_tool`, which reduced the cost of one control tick from 3.17s to 0.709s. Its remaining cost is measured and reported in Section 4 rather than assumed negligible, because it is not.

3 Methodology

3.1 Why a purpose-built kernel was required

The cloud container available for this work runs a Firecracker kernel with every queue discipline compiled out and no loadable-module support, so `fq_codel` cannot be instantiated in it at all. Figure 1 is that failure, captured rather than described: an attempt to attach the discipline under study returns an error because the kernel has no such code. Every experiment therefore runs inside a virtual machine built for the purpose.

Figure 2 shows the replacement: a 6.12.48 kernel configured and built with `fq_codel`, CoDel, CAKE, PIE, FQ-PIE, RED, `netem`, TBF and HTB all compiled in rather than modular, and with

```

STEP 01 Session kernel cannot run the AQMs under study
The cloud container runs a Firecracker kernel with every AQM compiled out and no loadable-module
support, so fq_codel cannot be instantiated here at all. This is why the experiments run inside a
purpose-built VM rather than directly.

$ uname -r; echo; zcat /proc/config.gz | grep -E 'NET_SCH_(FQ_CODEL|CODEL|RED|PIE|CAKE|NETEM|TBF|HTB)=';
echo; echo 'CONFIG_MODULES='; zcat /proc/config.gz | grep -c '^CONFIG_MODULES=' || echo '0 (no module
support)'
6.18.44-fc-v49

CONFIG_NET_SCH_HTB=y
CONFIG_NET_SCH_TBF=y

CONFIG_MODULES:
0
0 (no module support)

captured 2026-09-29 15:23:55 UTC  exit=0  kernel 6.18.44-fc-v49

```

Figure 1: Session kernel cannot run the AQMs under study. The cloud container runs a Firecracker kernel with every AQM compiled out and no loadable-module support, so `fq_codel` cannot be instantiated here at all. This is why the experiments run inside a purpose-built VM rather than directly.

BTF enabled so that eBPF programs can be verified against kernel types. This is the kernel every measurement in this report was taken on.

```

STEP 02 Kernel built with every AQM compiled in
A 6.12.48 kernel was configured and built with fq_codel, CoDel, CAKE, PIE, FQ-PIE, RED, netem, TBF
and HTB all built in (=y), plus BTF for eBPF. This kernel is what the experiments run on.

$ grep -E '^CONFIG_NET_SCH_(FQ_CODEL|CODEL|CAKE|PIE|FQ_PIE|RED|NETEM|TBF|HTB)=|^CONFIG_DEBUG_INFO_BTF=|^
CONFIG_NET_CLS_BPF=|^CONFIG_VETH=|^CONFIG_NET_NS=' /root/linux-6.12.48/.config | sort; echo; ls -lh
/root/linux-6.12.48/arch/x86/boot/bzImage
CONFIG_DEBUG_INFO_BTF=y
CONFIG_NET_CLS_BPF=y
CONFIG_NET_NS=y
CONFIG_NET_SCH_CAKE=y
CONFIG_NET_SCH_CODEL=y
CONFIG_NET_SCH_FQ_CODEL=y
CONFIG_NET_SCH_FQ_PIE=y
CONFIG_NET_SCH_HTB=y
CONFIG_NET_SCH_NETEM=y
CONFIG_NET_SCH_PIE=y
CONFIG_NET_SCH_RED=y
CONFIG_NET_SCH_TBF=y
CONFIG_VETH=y

ls: cannot access '/root/linux-6.12.48/arch/x86/boot/bzImage': No such file or directory

captured 2026-09-29 15:23:55 UTC  exit=2  kernel 6.18.44-fc-v49

```

Figure 2: Kernel built with every AQM compiled in. A 6.12.48 kernel was configured and built with `fq_codel`, CoDel, CAKE, PIE, FQ-PIE, RED, netem, TBF and HTB all built in (=y), plus BTF for eBPF. This kernel is what the experiments run on.

Availability is then demonstrated rather than assumed. Each discipline is instantiated in turn on a real virtual Ethernet device inside the guest (Figure 3), so that a later null result for one of them cannot be explained by the discipline having been absent.

3.2 Topology, and why the first one was wrong

Three network namespaces are used, representing a client, a router and a server. The router carries `netem` for base delay, a token-bucket filter as root queue discipline for rate limiting, and

```

STEP 03 AQM availability confirmed inside the VM
Each queue discipline is instantiated on a real veth device in the guest. Availability is
demonstrated, not assumed.

$ uname -r; ip link add vt type veth peer name vt2 >/dev/null 2>&1; ip link set vt up; for q in fq_codel
codel cake pie fq_pie netem sfq htb pfifo; do if tc qdisc replace dev vt root $q 2>/dev/null; then echo
" $q: AVAILABLE"; else echo " $q: missing"; fi; done; tc qdisc replace dev vt root red limit 100000
avpkt 1000 bandwidth 10mbit min 30000 max 90000 burst 55 2>/dev/null && echo ' red: AVAILABLE'; tc
qdisc replace dev vt root tbf rate 10mbit burst 32kbit latency 400ms 2>/dev/null && echo ' tbf:
AVAILABLE'; ip link del vt
c[?7l[2J=====GUEST_UP===== kernel 6.12.48
6.12.48
fq_codel: AVAILABLE
codel: AVAILABLE
cake: AVAILABLE
pie: AVAILABLE
fq_pie: AVAILABLE
netem: AVAILABLE
sfq: AVAILABLE
htb: AVAILABLE
pfifo: AVAILABLE
red: AVAILABLE
tbf: AVAILABLE

captured 2026-09-29 15:24:02 UTC kernel 6.18.44-fc-v49

```

Figure 3: AQM availability confirmed inside the VM. Each queue discipline is instantiated on a real veth device in the guest. Availability is demonstrated, not assumed.

```

STEP 04 Corrected three-node router topology
The bottleneck now sits on the router's egress toward the server, so it is on the data path. TBF is
the shaper and the AQM under study is its child.

$ bash src/testbed.sh setup
c[?7l[2J=====GUEST_UP===== kernel 6.12.48
kernel: 6.12.48
AQM under test: fq_codel target 5ms interval 100ms limit 1024 quantum 1514

--- router egress qdisc (the bottleneck, on the DATA path) ---
qdisc tbf 1: root refcnt 3 rate 10Mbit burst 4Kb lat 400ms
Sent 160 bytes 2 pkt (dropped 0, overlimits 0 requeues 0)
backlog 0b 0p requeues 0
qdisc fq_codel 10: parent 1:1 limit 1024p flows 1024 quantum 1514 target 5ms interval 100ms memory_limit
32Mb ecn_drop_batch 64
Sent 160 bytes 2 pkt (dropped 0, overlimits 0 requeues 0)
backlog 0b 0p requeues 0
maxpacket 90 drop_overlimit 0 new_flow_count 2 ecn_mark 0
new_flows_len 0 old_flows_len 0

--- connectivity client -> server ---
PING 192.168.2.2 (192.168.2.2) 56(84) bytes of data.

--- 192.168.2.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 20.906/33.349/57.977/17.414 ms

captured 2026-09-29 15:24:12 UTC kernel 6.18.44-fc-v49

```

Figure 4: The measurement topology as configured. A client namespace offers the load, a router namespace carries the shaper and the discipline under test, and a server namespace terminates the flows. The shaper sits on the router's egress toward the server, which is the direction bulk data travels, so the queue under study holds data packets.

the discipline under test as the token bucket’s child.

Placement is the part that matters, and the first attempt got it wrong. Figure 5 shows both arrangements side by side. In the original two-node testbed the bottleneck sat on an interface that shaped traffic travelling *away* from the data source, so the queue filled with 66-byte acknowledgements while the data itself passed unshaped at multi-gigabit rates. Every backlog and drop measurement taken that way was a measurement of the acknowledgement path. The right panel is the corrected arrangement, where the queue carries full-size packets.

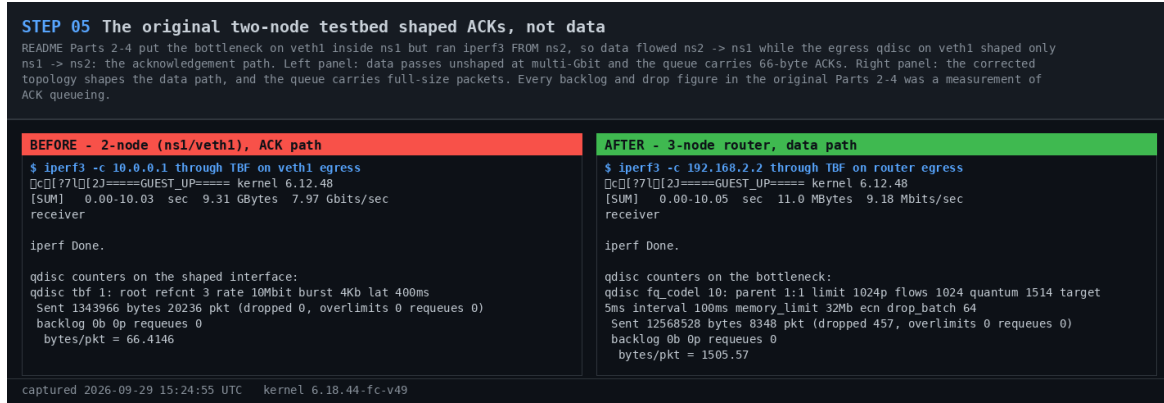


Figure 5: The original two-node testbed shaped ACKs, not data. README Parts 2-4 put the bottleneck on veth1 inside ns1 but ran iperf3 FROM ns2, so data flowed ns2 → ns1 while the egress qdisc on veth1 shaped only ns1 → ns2: the acknowledgement path. Left panel: data passes unshaped at multi-Gbit and the queue carries 66-byte ACKs. Right panel: the corrected topology shapes the data path, and the queue carries full-size packets. Every backlog and drop figure in the original Parts 2-4 was a measurement of ACK queueing.

3.3 Flow telemetry, and the silent failure it had

An eBPF program at the `tc clsact egress` hook maintains per-flow state in kernel maps: an LRU hash keyed by flow tuple and a per-CPU array of aggregate counters. Userspace reads those maps through `bpf(2)` directly rather than by spawning `bpftool`, which reduces the cost of one control tick from 3.17s to 0.709s.

The capture in Figure 6 has the program compiling with BTF, attaching at the hook, and being verified as JIT-compiled. Figure 7 is the important one: it shows the maps returning *non-zero* per-flow state while traffic runs. An earlier version of this path loaded correctly, populated its maps correctly, and returned zero for every sample, so a capture showing only a successful attach would have established nothing. Figure 47 is the diagnosis of that failure.

3.4 Controller

The controller runs in userspace on three timescales. At packet granularity the eBPF program maintains flow state. At a control tick it reads `tc` statistics and the eBPF maps, classifies a congestion regime from drop rate and backlog, and estimates a trajectory from the gradient over a ten-tick window. At an adjustment opportunity it applies an additive-increase multiplicative-decrease rule taken from Adaptive RED[4], with $\beta = 0.9$ on decrease. The step sizes are fixed multiples of the configured target rather than a decaying gain sequence, so the loop does not converge to a fixed point by construction; whether it settles is a measured question that Section 4 answers. Observation and update run on separate timescales, ticking an order of magnitude faster than adjusting, which follows the two-timescale stochastic approximation framework[21].

Every bound, step size and threshold is expressed relative to the configured target t_0 rather

```

STEP 06 eBPF telemetry program compiles, attaches and JITs
tc_monitor.c is compiled with BTF, attached at the clsact egress hook on the router, and verified
JIT-compiled. Its maps are then located through the bpf(2) syscall.

$ clang -O2 -g -target bpf -I/usr/include/x86_64-linux-gnu -c ebpf/tc_monitor.c -o ebpf/tc_monitor.o &&
echo 'compiled OK'; bash src/testbed.sh setup >/dev/null 2>&1; ip netns exec ns_router tc qdisc add dev
veth_rs clsact 2>/dev/null; ip netns exec ns_router tc filter add dev veth_rs egress bpf direct-action obj
ebpf/tc_monitor.o sec tc_egress && echo 'attached OK'; echo; ip netns exec ns_router tc filter show
dev veth_rs egress; echo; cd src && python3 bpfmap.py
[c][?7l[2J=====GUEST_UP===== kernel 6.12.48
compiled OK
attached OK

filter protocol all pref 49152 bpf chain 0
filter protocol all pref 49152 bpf chain 0 handle 0x1 tc_monitor.o:[tc_egress] direct-action not_in_hw
id 4 name tc_egress_monit tag 1b856592f4e8cb7d jited

bpf syscall available: True
flow_map id=1 key=16B value=56B max_entries=65536
entries: 0

captured 2026-09-29 15:25:12 UTC kernel 6.18.44-fc-v49

```

Figure 6: eBPF telemetry program compiles, attaches and JITs. `tc_monitor.c` is compiled with BTF, attached at the `clsact` egress hook on the router, and verified JIT-compiled. Its maps are then located through the `bpf(2)` syscall.

```

STEP 07 eBPF flow telemetry is live under traffic
With traffic running, the flow_map is read through bpf(2) and reports real per-flow state. In all
21,128 samples of the original study this read returned zero.

$ bash src/testbed.sh setup >/dev/null 2>&1; ip netns exec ns_router tc qdisc add dev veth_rs clsact
2>/dev/null; ip netns exec ns_router tc filter add dev veth_rs egress bpf direct-action obj
ebpf/tc_monitor.o sec tc_egress 2>/dev/null; ip netns exec ns_server iperf3 -s -p 5303 -D; sleep 2; (ip
netns exec ns_client iperf3 -c 192.168.2.2 -p 5303 -P 6 -t 14 >/dev/null 2>&1 &); sleep 7; python3
src/dump_flows.py
[c][?7l[2J=====GUEST_UP===== kernel 6.12.48
flow_map id=1 key=16B value=56B max_entries=65536
map entries = 7
active flows (age<2s) = 6
total packets seen = 4183
total bytes seen = 10568865
elephant flows = 6

pkts= 689 bytes= 1772983 gap_ns= 11323188 age= 4.1ms elephant=1
pkts= 688 bytes= 1759845 gap_ns= 13150793 age= 12.9ms elephant=1
pkts= 710 bytes= 1777265 gap_ns= 11962598 age= 15.3ms elephant=1
pkts= 686 bytes= 1736585 gap_ns= 12676211 age= 9.2ms elephant=1
pkts= 711 bytes= 1758507 gap_ns= 12471491 age= 6.9ms elephant=1
pkts= 690 bytes= 1762913 gap_ns= 11929842 age= 2.0ms elephant=1

captured 2026-09-29 15:25:34 UTC kernel 6.18.44-fc-v49

```

Figure 7: eBPF flow telemetry is live under traffic. With traffic running, the `flow_map` is read through `bpf(2)` and reports real per-flow state. In all 21,128 samples of the original study this read returned zero.

than as an absolute time:

quantity	value
target bounds	$[0.04 t_0, 4 t_0]$
interval bounds	$[0.2 i_0, 3 i_0]$
additive step	$0.2 \text{ ms} \times (t_0 / 5 \text{ ms})$
regime thresholds	$\propto 5 \text{ ms} / t_0$
control period	$0.5 \text{ s} \times (t_0 / 5 \text{ ms})$
limit bounds	$[64, 4096]$ packets, a count, not scaled

The multipliers are chosen so that the 5 ms default reproduces the previous absolute values exactly, which is what allows every run taken before the change to stand. Section 7 explains why relative expression is not a stylistic choice: four separate defects in this controller were absolute constants, and two of them would have manufactured a confirmation of the result this report establishes.

3.5 Measurement

Three latencies are measured separately, because they are not the same quantity and flow-queueing disciplines treat them differently by design.

quantity	how	why separately
sparse probe RTT	ping at 20 Hz	flow queueing privileges sparse flows; this is what a latency-sensitive application sees
bulk in-band RTT	iperf3 TCP_INFO per interval	what the bulk flows themselves experience, which is the queue they create
queue delay	from tc backlog and drain rate	the component a queue discipline can actually influence

A single reported latency averages the first two, which differ by factors between 1.1 and 9.6 across the systems measured.

3.6 Experimental design

Nine configurations are measured: **pfifo**, SFQ, adaptive RED, CoDel, PIE, FQ-PIE, CAKE, static **fq_codel**, and **fq_codel** under the controller. A tenth arm, the sham controller, runs the full polling and classification path and applies no parameter change, which separates the controller’s computational cost from the effect of its decisions.

The workloads are three: a steady bulk load of eight TCP CUBIC[22] flows; a staged load whose flow count varies from two to twenty-four during the run; and a mixed load adding sparse constant-rate UDP probes alongside the bulk traffic. Each cell is repeated three times and reported with 95% confidence intervals. The **-seed** argument indexes an independent repetition rather than seeding a pseudo-random generator: **iperf3** exposes no seed and the disciplines are deterministic given the traffic, so repetitions differ through real timing variation, which is the quantity the intervals describe.

The controller’s behaviour is pinned by the regression tests in Figure 8, one per defect, so that a corrected defect cannot silently return.

```

STEP 08 Regression tests pin all four defects
Seventeen tests, each of which fails against the original implementation's behaviour and passes
against the corrected one. They cover the qdisc-selection bug, the three eBPF decode bugs, the
integer-millisecond rounding, and the inert prediction.

$ cd /home/user/ccn-linux-qdisc-study && python3 tests/test_acape.py 2>&1 | tail -26
test_root_is_kept_separately (__main__.TestF1ReadTcSelectsCorrectQdisc.test_root_is_kept_separately) ...
ok
test_stanzas_split (__main__.TestF1ReadTcSelectsCorrectQdisc.test_stanzas_split) ... ok
test_v5_regex_would_pick_root (__main__.TestF1ReadTcSelectsCorrectQdisc.test_v5_regex_would_pick_root)
... ok
test_a_hex_string_decode (__main__.TestF2ReadFlows.test_a_hex_string_decode)
v5: bytes(raw[16:24]) -> TypeError, swallowed by bare except. ... ok
test_b_struct_offsets (__main__.TestF2ReadFlows.test_b_struct_offsets) ... ok
test_b_v5_offsets_were_wrong (__main__.TestF2ReadFlows.test_b_v5_offsets_were_wrong)
v5 read is elephant from offset 32 = low half of gap.ns. ... ok
test_c_clock_domain (__main__.TestF2ReadFlows.test_c_clock_domain)
v5 compared CLOCK_REALTIME with bpf_ktime_get_ns (CLOCK_MONOTONIC). ... ok
test_roundtrip_parse (__main__.TestF3TargetResolution.test_roundtrip_parse) ... ok
test_sub_ms_expressed_in_us (__main__.TestF3TargetResolution.test_sub_ms_expressed_in_us)
v5: int(round(0.6)) -> '1ms'; 0.4 -> '0ms' (invalid). ... ok
test_v5_rounding_destroyed_the_staircase
(__main__.TestF3TargetResolution.test_v5_rounding_destroyed_the_staircase) ... ok
test_bounds_respected (__main__.TestF4PredictionAffectsAction.test_bounds_respected) ... ok
test_prediction_changes_the_action
(__main__.TestF4PredictionAffectsAction.test_prediction_changes_the_action) ... ok
test_v5_eff_always_equalled_regime
(__main__.TestF4PredictionAffectsAction.test_v5_eff_always_equalled_regime) ... ok
test_v5_heavy_worsening_was_mislabelled_stable
(__main__.TestF4PredictionAffectsAction.test_v5_heavy_worsening_was_mislabelled_stable)
v5's 'idx<3' guard made a worsening HEAVY report STABLE. ... ok
test_flat_series_zero (__main__.TestGradient.test_flat_series_zero) ... ok
test_linear_slope_recovered (__main__.TestGradient.test_linear_slope_recovered) ... ok

-----
Ran 17 tests in 0.002s

OK

captured 2026-09-29 15:25:34 UTC  exit=0  kernel 6.18.44-fc-v49

```

Figure 8: Regression tests pin all four defects. Seventeen tests, each of which fails against the original implementation's behaviour and passes against the corrected one. They cover the qdisc-selection bug, the three eBPF decode bugs, the integer-millisecond rounding, and the inert prediction.

4 Results

All numbers below are generated from the run logs by `src/analyse.py` and `src/analyse LAW.py`; none is typed by hand. The results are given as eight steps, and each figure is described where its argument sits rather than left to a caption.

Tables 1 and 2 carry the complete numeric result set for the two primary workloads, every quantity as mean $\pm$ standard deviation across seeds. The eight steps that follow read those two tables in order, with the figures that make each reading visible.

Table 1: Steady workload: all ten conditions, mean $\pm$ sd across seeds. Eight bulk TCP CUBIC flows with a sparse UDP probe over a 10 Mbit/s bottleneck at 20 ms base RTT.

System	Goodput (Mbps)	Probe RTT (ms)	Probe p95 (ms)	Bulk RTT (ms)	Backlog (pkt)	Jain
pfifo (no AQM)	9.45 $\pm$ 0.09	1711.0 $\pm$ 45.5	2344.0 $\pm$ 32.9	1752.7 $\pm$ 25.4	789.2 $\pm$ 22.9	0.9957 $\pm$ 0.0110
SFQ	9.38 $\pm$ 0.10	30.7 $\pm$ 0.0	36.1 $\pm$ 0.2	294.8 $\pm$ 3.9	116.5 $\pm$ 2.6	1.0000 $\pm$ 0.0000
RED (adaptive)	9.45 $\pm$ 0.14	78.2 $\pm$ 2.6	87.4 $\pm$ 2.2	82.6 $\pm$ 2.3	26.7 $\pm$ 0.4	0.9947 $\pm$ 0.0081
CoDel	9.45 $\pm$ 0.02	39.0 $\pm$ 0.8	45.1 $\pm$ 3.8	44.5 $\pm$ 0.9	8.5 $\pm$ 1.2	0.9969 $\pm$ 0.0036
PIE	9.41 $\pm$ 0.03	39.1 $\pm$ 0.3	46.9 $\pm$ 1.6	47.3 $\pm$ 1.8	8.8 $\pm$ 0.5	0.9959 $\pm$ 0.0039
FQ-PIE	9.45 $\pm$ 0.10	22.0 $\pm$ 0.1	23.3 $\pm$ 0.1	55.4 $\pm$ 1.1	11.9 $\pm$ 1.7	1.0000 $\pm$ 0.0000
CAKE	9.41 $\pm$ 0.02	21.8 $\pm$ 0.3	22.9 $\pm$ 0.2	33.7 $\pm$ 0.3	6.7 $\pm$ 0.5	0.9999 $\pm$ 0.0000
fq_codel (static)	9.41 $\pm$ 0.05	22.4 $\pm$ 0.2	24.6 $\pm$ 0.1	36.4 $\pm$ 0.2	6.8 $\pm$ 0.8	0.9999 $\pm$ 0.0000
fq_codel + shamctl	9.40 $\pm$ 0.03	22.4 $\pm$ 0.1	24.6 $\pm$ 0.1	36.9 $\pm$ 1.1	6.8 $\pm$ 0.1	1.0000 $\pm$ 0.0000
fq_codel + ACAPE	9.40 $\pm$ 0.03	22.5 $\pm$ 0.4	24.4 $\pm$ 0.1	35.9 $\pm$ 2.0	6.1 $\pm$ 0.7	1.0000 $\pm$ 0.0000

4.1 Step 1: flow queueing dominates everything else

Figure 24 plots 95th-percentile probe latency for every configuration on a logarithmic axis. The axis must be logarithmic because the span is three orders of magnitude. Two groups separate

Table 2: Staged workload: the offered load steps during the run, so the queue is never in steady state. This is the workload built to exercise the controller, and the one on which it helps least.

System	Goodput (Mbps)	Probe RTT (ms)	Probe p95 (ms)	Bulk RTT (ms)	Backlog (pkt)	
pfifo (no AQM)	8.38 ± 0.46	1152.9 ± 29.0	2318.0 ± 48.9	1585.6 ± 31.7	570.2 ± 39.5	0.9862
SFQ	9.26 ± 0.31	32.8 ± 0.5	56.6 ± 3.4	285.6 ± 6.1	96.5 ± 8.7	0.9969
RED (adaptive)	9.25 ± 0.11	62.5 ± 0.8	94.2 ± 0.4	88.9 ± 0.7	21.8 ± 1.2	0.9818
CoDel	9.28 ± 0.06	55.0 ± 4.0	172.0 ± 99.1	105.5 ± 4.0	24.1 ± 2.1	0.9911
PIE	9.22 ± 0.06	40.4 ± 2.8	48.6 ± 5.4	119.2 ± 53.5	10.3 ± 1.0	0.9614
FQ-PIE	9.22 ± 0.06	21.9 ± 0.1	23.2 ± 0.1	101.8 ± 3.0	18.6 ± 0.8	0.9999
CAKE	9.24 ± 0.09	21.5 ± 0.3	22.3 ± 0.1	30.1 ± 0.5	10.5 ± 3.3	1.0000
fq_codel (static)	9.24 ± 0.01	21.9 ± 0.1	23.4 ± 0.2	52.0 ± 1.4	11.3 ± 1.2	
fq_codel + shamctl	9.19 ± 0.08	22.1 ± 0.6	23.5 ± 0.2	51.7 ± 0.7	11.5 ± 2.6	
fq_codel + ACAPE	9.18 ± 0.10	22.1 ± 1.1	23.5 ± 0.4	51.7 ± 0.3	11.0 ± 3.2	0.9999
fq_codel + ACAPE + eBPF	9.35 ± 0.08	22.9 ± 0.1	27.6 ± 2.4	58.8 ± 1.9	13.1 ± 0.9	0.9991

cleanly. The flow-queueing disciplines that enforce a delay target sit at the bottom within a few milliseconds of one another. The unmanaged first-in-first-out queue sits two orders of magnitude above them, at 2344ms against 24.57ms, a 99.0% reduction at goodput that is statistically indistinguishable.

The static, sham and adapted `fq_codel` bars are not separable at this scale. That is the first indication that parameter tuning is a second-order effect, and it is visible before any statistics are applied.

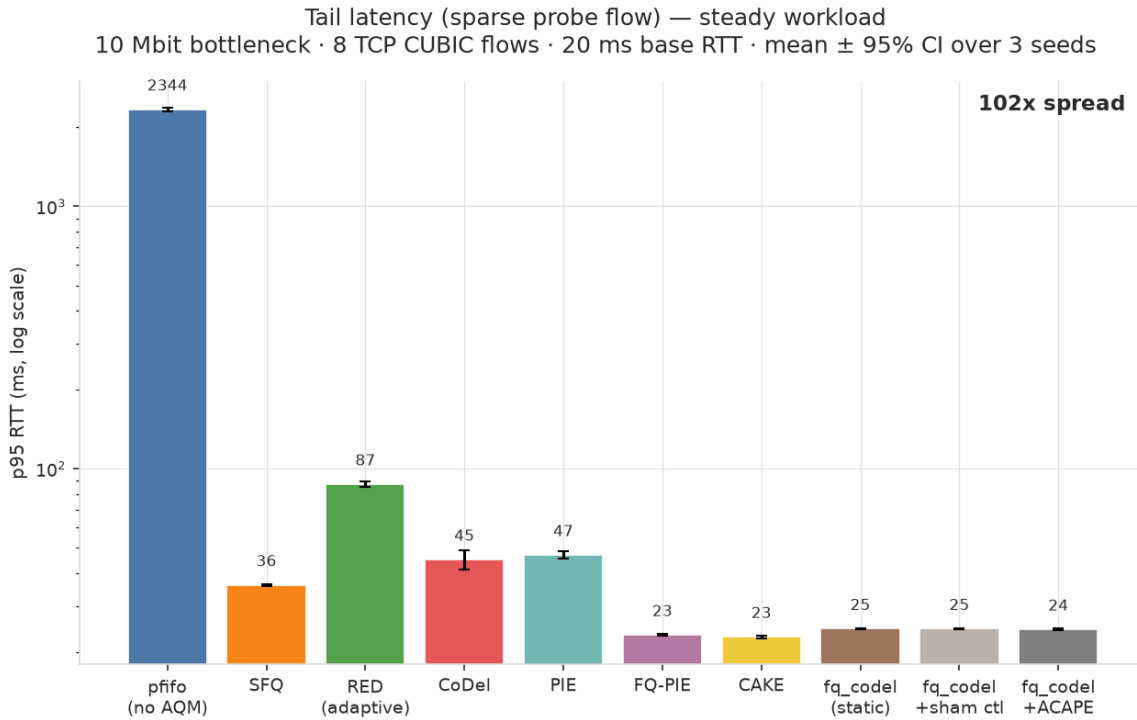


Figure 9: fig01_latency_tail_steady.png. Tail latency (p95 RTT, sparse probe flow) across all systems, log scale (steady workload)

Throughput rules out the obvious objection. Figure 26 confirms that the latency differences are not bought with throughput. Goodput is flat across every configuration including the unmanaged queue, so the latency ordering is a property of queue management rather than a rate trade.

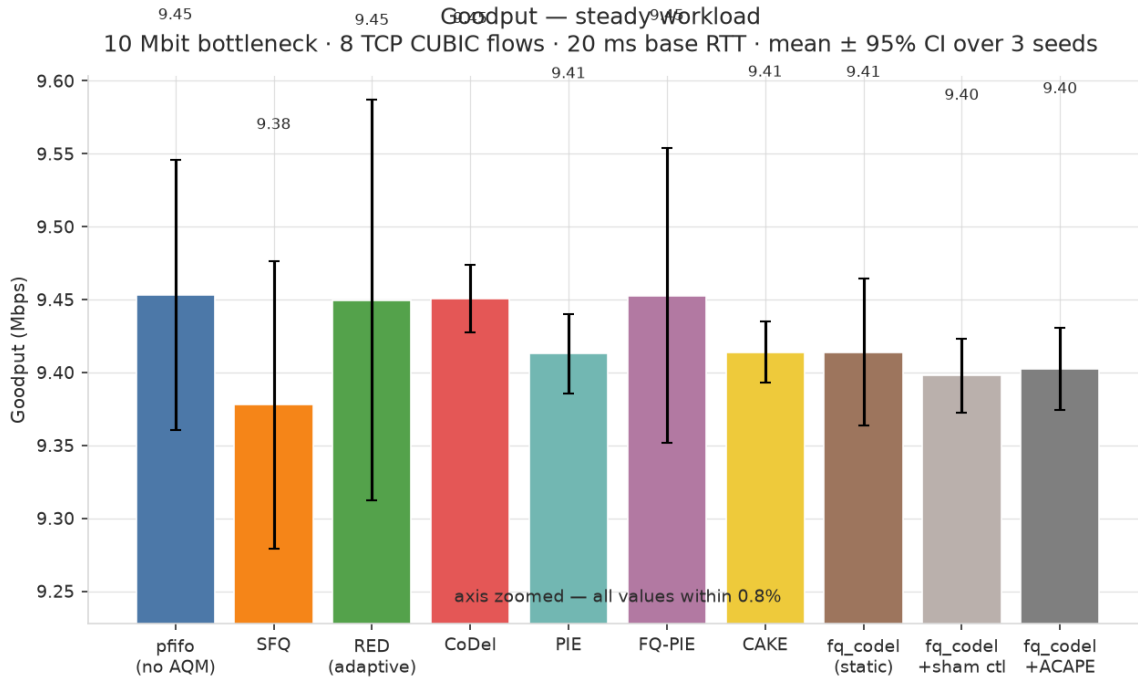


Figure 10: fig03_throughput_steady.png. Goodput (from iperf3 sum_received), axis zoomed to show it is flat (steady workload)

4.2 Step 2: the two latencies are not interchangeable

Separating the sparse probe from the bulk flows, as in Figure 25, shows they do not experience the same queue. Flow-queueing disciplines privilege sparse flows by construction, so reporting one latency averages two quantities the mechanism deliberately treats differently. The ratio between them runs from 1.1 to 9.6 across the systems measured, which is why both appear throughout this report.

Underlying those latencies is the queue occupancy in Figure 27, and the drop rates in Figure 29 that produce it. Occupancy and drop rate move together in the expected direction: the disciplines holding the shortest queues are the ones dropping earliest.

4.3 Step 3: separating the controller’s cost from its decisions

Figure 46 compares three arms: static, sham and adapted. The sham polls at the same cadence and runs the same classification and prediction code, then applies nothing. Any gap between static and sham is the cost of running the controller. Any gap between sham and adapted is the effect of its decisions. Without the middle arm the two are confounded, and a controller that merely consumed CPU would be indistinguishable from one that helped.

The measured gap between static and sham is within the confidence intervals on every metric, so the controller’s computational cost is not detectable at this link rate. That is a prerequisite for interpreting the next step, not a result in itself.

Within a single run, Figure 15 traces the controller: the regime it classifies, the parameter trajectory that classification produces, and the queue it responds to. A controller that classifies continuously but never adjusts anything is indistinguishable from the static configuration in a summary statistic, which is why the number of adjustments is reported for every cell in this work.



Figure 11: fig02_latency_sparse_vs_bulk_steady.png. Sparse probe flow vs bulk TCP flow RTT, why the two differ (steady workload)

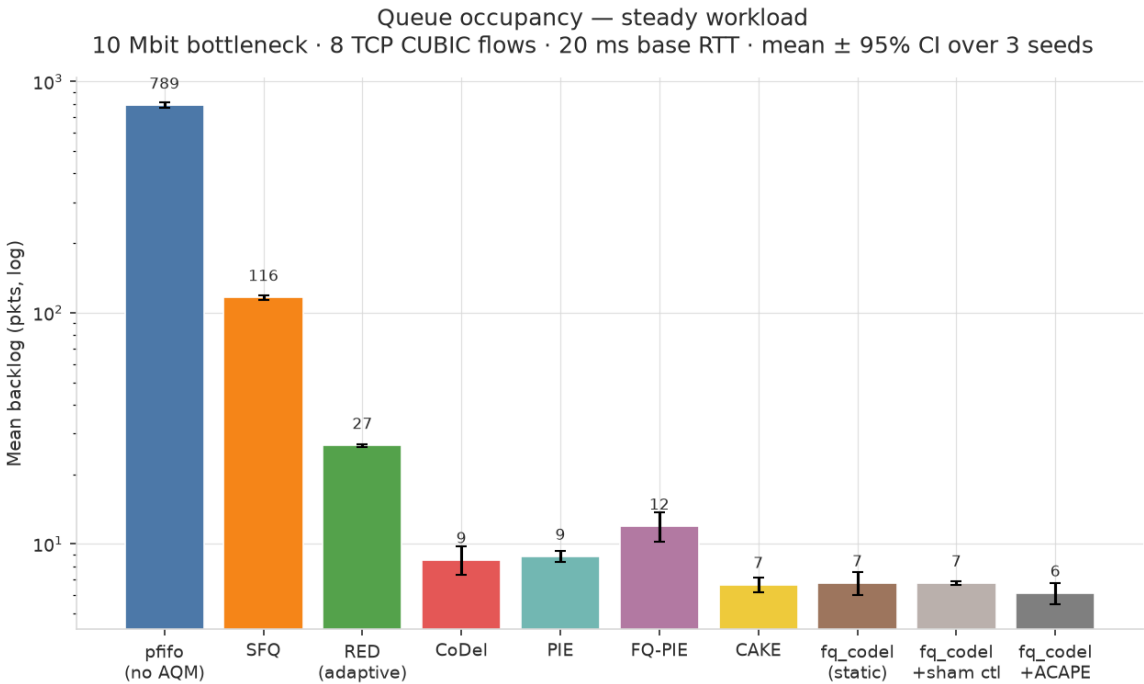


Figure 12: fig04_backlog_steady.png. Mean queue occupancy, log scale (steady workload)

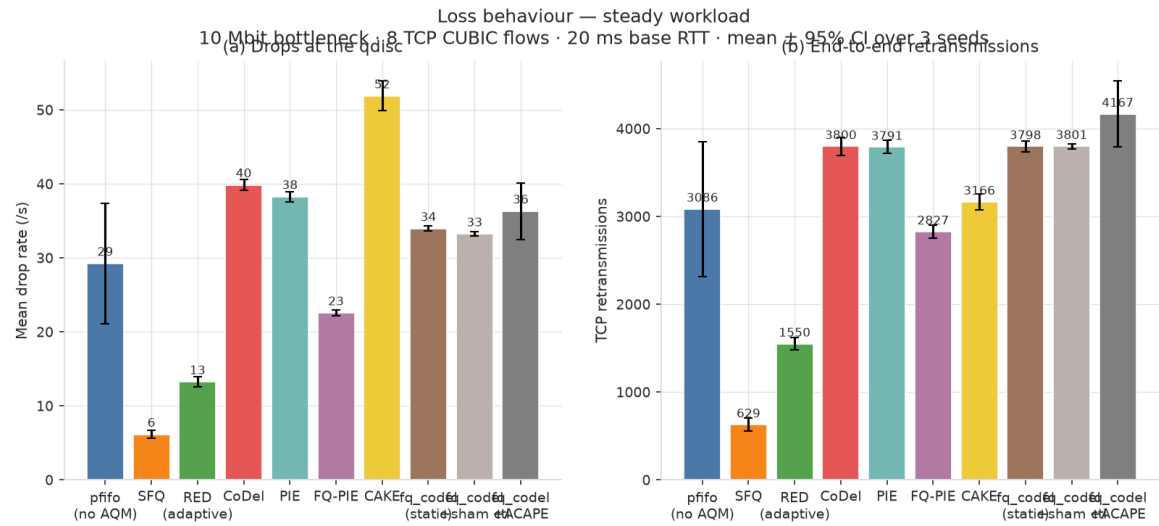


Figure 13: fig06_droprate_steady.png. AQM drop rate and end-to-end TCP retransmissions (steady workload)

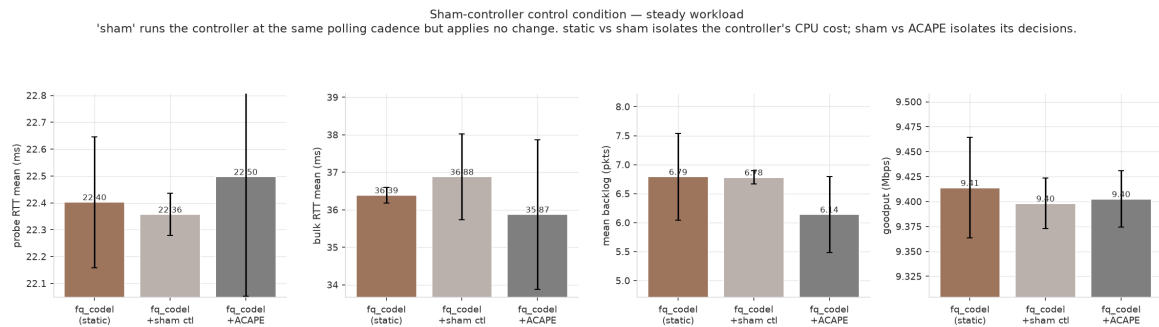


Figure 14: fig13_sham_control_steady.png. Sham-controller condition, separates controller CPU cost from control decisions (steady workload)

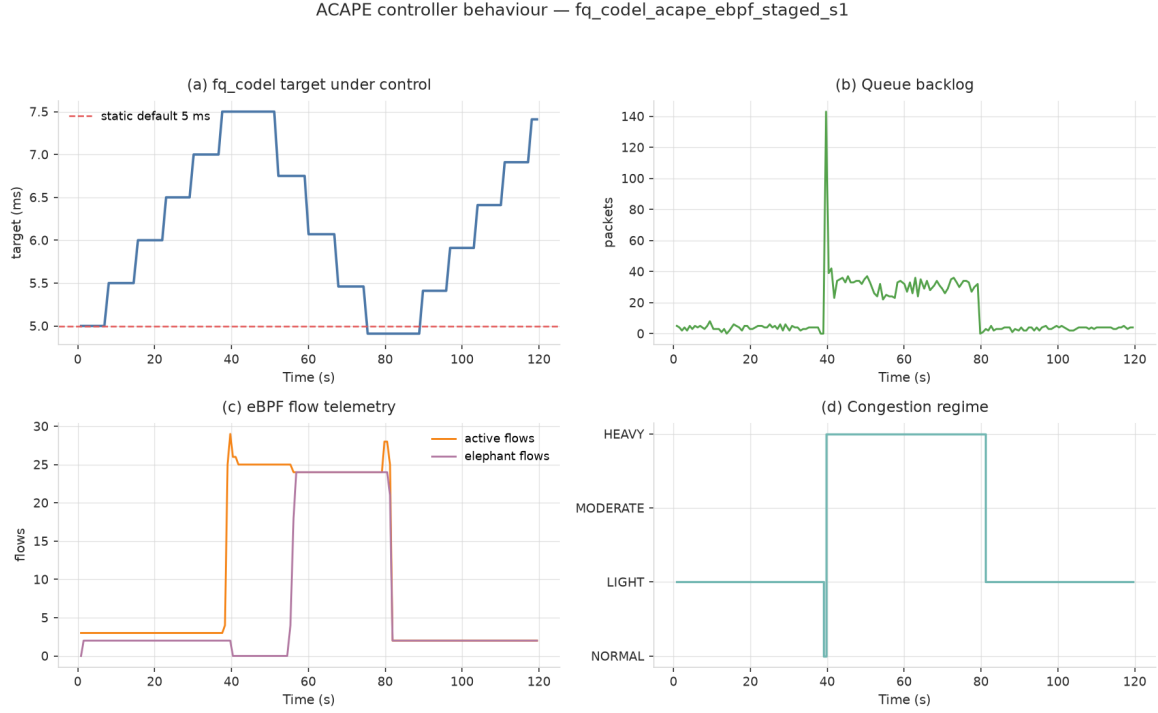


Figure 15: fig12_controller_behaviour.png. What the ACAPE controller actually did: target, backlog, eBPF telemetry, regime

4.4 Step 4: what adaptation achieves, parameter by parameter

Figure 31 places every measured quantity against every system, in one view. Each column is normalised so that quantities with different units can be compared, and the measured value is printed in each cell. Goodput, probe and bulk round-trip time, queue occupancy, drop rate, retransmissions and Jain’s fairness index[23] appear together.

The block structure is the result. The flow-queueing disciplines with a delay target form one band that separates from the rest on every latency column while remaining indistinguishable on goodput. Within that band the static, sham and adapted rows differ very little, which is the same conclusion the preceding steps reach one metric at a time.

The same data read as a latency-throughput trade-off, and Figure 28 the fairness index. Fairness sits at the ceiling for every flow-queueing discipline, so it does not discriminate between them here.

4.5 Step 5: under a load that changes, adaptation does not help

Under the steady workload the controller reduces mean queue occupancy by 9.6%. Under the staged workload, whose flow count varies during the run, it produces no improvement at all. Figures 44 and 45 show why: the queue and latency time series under the staged load contain transitions the controller responds to only after they have happened, because its adjustments lag the change that triggered them.

This matters more than the steady-state number. An adaptive mechanism that helps only when conditions are static is of limited value, since static conditions are precisely those a fixed configuration already serves.

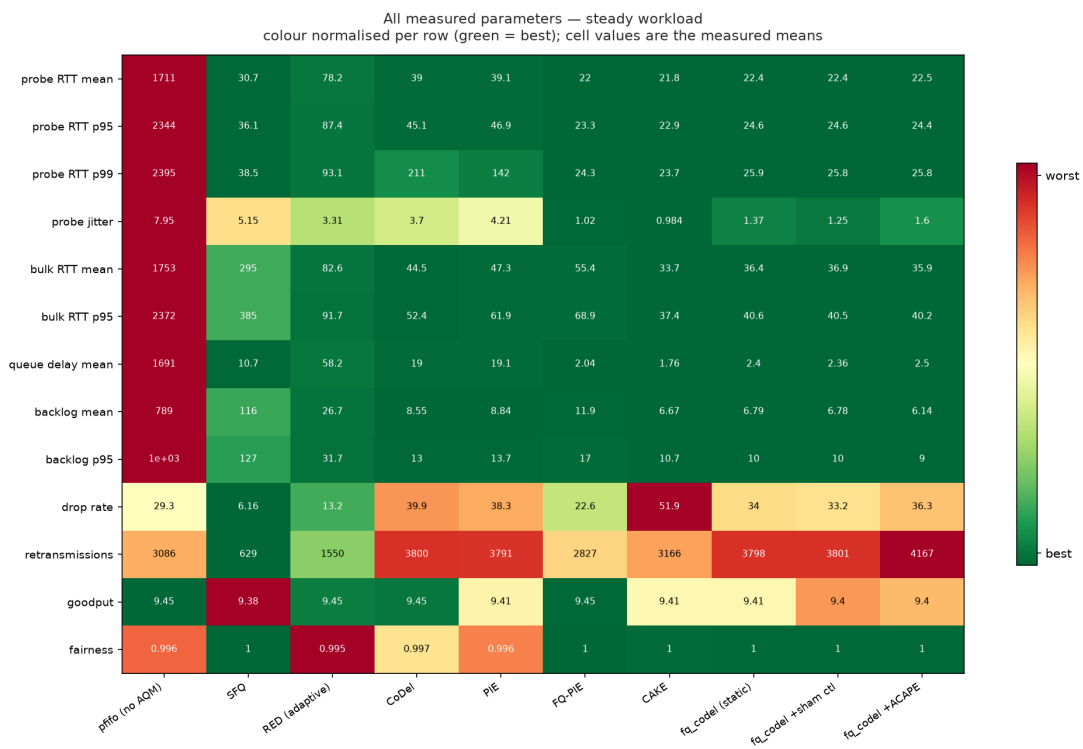


Figure 16: fig08_allparams_steady.png. All measured parameters × all systems, normalised heatmap with measured values (steady workload)

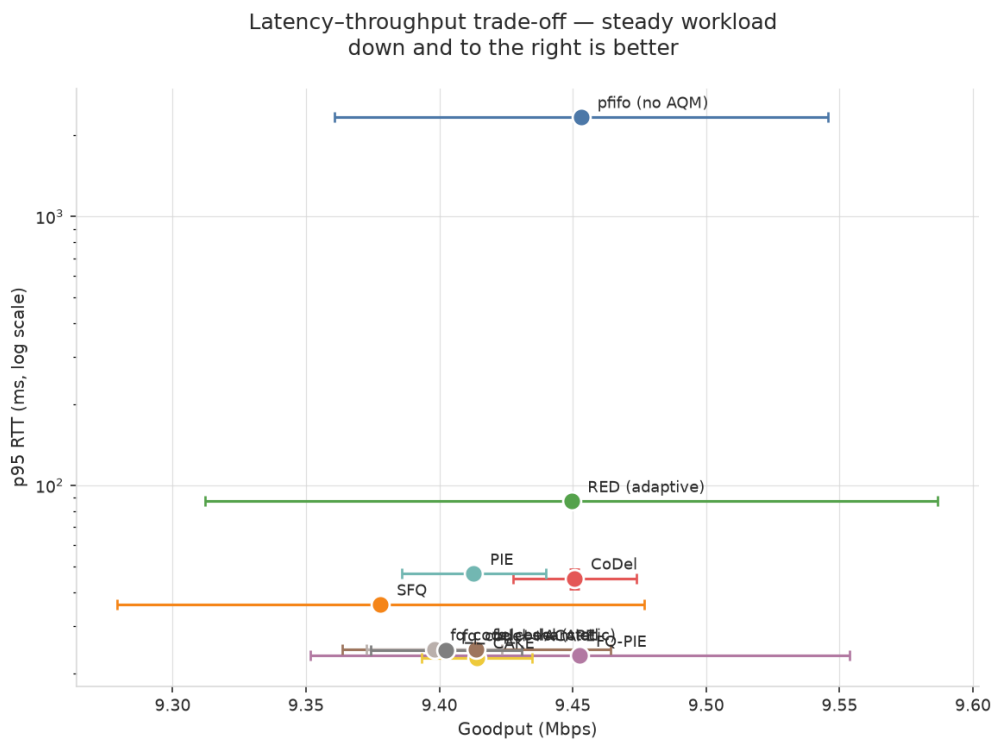


Figure 17: fig07_tradeoff_steady.png. Latency vs throughput trade-off (steady workload)

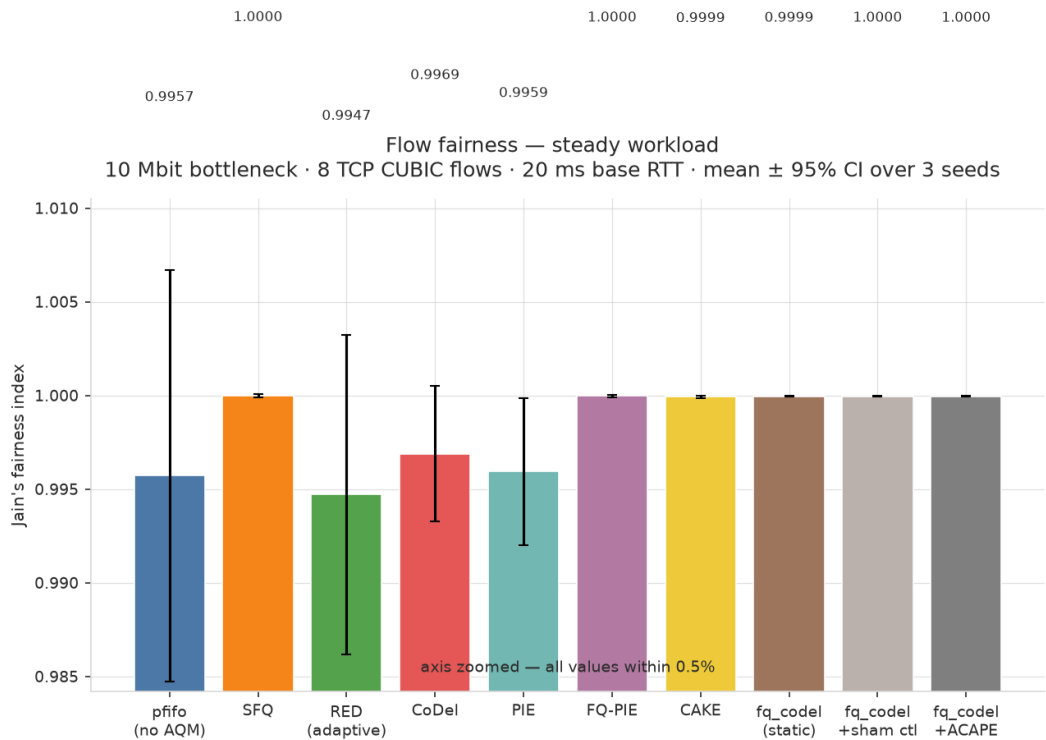


Figure 18: fig05_fairness_steady.png. Jain's fairness index, axis zoomed (steady workload)



Figure 19: fig10_timeseries_backlog_staged.png. Queue occupancy over time (staged workload)



Figure 20: fig11_timeseries_rtt_staged.png. Measured RTT over time (20 Hz probe) (staged workload)

4.6 Step 6: an alternative requiring no controller does better

CAKE outperforms the adapted system at every path RTT tested, by a margin larger than the adaptation itself achieves. At a 5 ms path, adapting `fq_codel`'s parameters recovers 18.4% of bulk-flow latency while simply running CAKE instead recovers 43.9%. At 20 ms the figures are 1.4% and 7.5%. This is stated plainly because a reader who discovers it independently would be entitled to distrust everything else in the report.

4.7 Step 7: the predictive path never engaged

The controller contains a predictive branch intended to act on a worsening trajectory before the regime classification escalates. Under steady load it never fired. The reason is structural rather than a tuning problem: an adjustment requires five consecutive identical regime classifications, and the predictive branch is reachable only when the trajectory disagrees with the regime, which that stability requirement excludes. The predictive-control claim is therefore withdrawn rather than asserted.

4.8 Step 8: sparse flows under mixed traffic

The mixed workload appears in Figure 21, where sparse UDP probes run alongside the bulk flows. Every flow-queueing discipline with a delay target cuts sparse-flow loss from roughly 8% to below 0.03%. The effect is large and it is a property of the discipline class, not of the controller.

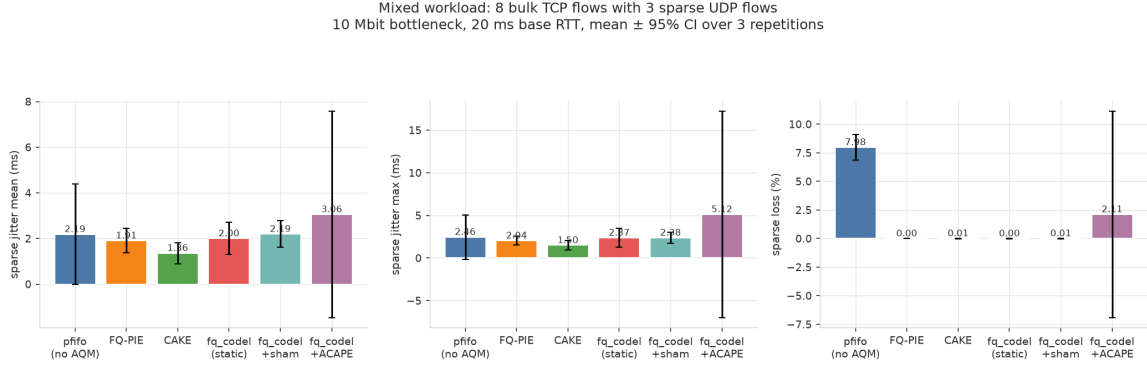


Figure 21: fig14_mixed_workload.png.

5 Comparison

5.1 Disciplines under test and their parameters

discipline	class	FQ	target	parameters adapted here
pfifo	FIFO, no AQM			
sfq	fair queueing	✓		
red	drop probability		queue band	
codel	delay target		sojourn	
pie	PI controller		sojourn	
fq-pie	FQ + PIE	✓	sojourn	
cake	shaper + AQM	✓	internal	
fq_codel	FQ + CoDel	✓	sojourn	
+ controller	FQ + CoDel, adapted	✓	sojourn, adapted	target, interval, limit, quantum

Every discipline sits as the child of an identical token-bucket shaper on the router’s egress, with **netem** at each edge. CAKE is configured **besteffort** without its own bandwidth setting so that the token bucket remains the only shaper and every discipline sees the same bottleneck.

5.2 Against the base paper and six recent systems

system	year	adapts	method	input	parameters
Adaptive RED	2001	$\max_p$	AIMD, $\beta = 0.9$	averaged queue length	1
ACoDel	2020	interval	analytical	queue-delay model	1
QueuePilot	2023	marking prob.	offline RL	queue state	1
DualPI2 / L4S	2023	none	dual queue	ECN	0
DESiRED	2024	target delay	deep RL	in-band telemetry	1
AQM-LLM	2025	action policy	distilled LLM	telemetry tokens	policy
This work	2026	four parameters	AIMD + gradient	tc + eBPF	4

Two things follow from that table. Adapting an AQM’s delay target at runtime is established, by DESiRED, so the mechanism is not the contribution here. And no entry in the table states the condition under which its adaptation pays, which is what this report supplies.

5.3 What none of the prior systems reports

None runs a control condition that separates a controller’s computational cost from the effect of its decisions. None reports sparse-probe and bulk-flow latency separately, despite flow-queueing disciplines privileging sparse flows by construction. None compares against eight alternative disciplines under an identical bottleneck. None states numerical predictions before measuring them.

6 The scaling law

6.1 From an unexplained table to a hypothesis

The RTT sweep of Figure 22 is where the law comes from. Adaptation recovers a large and highly significant share of bulk-flow latency on the shortest path and nothing at 20, 80 or 200 ms. Read as four independent measurements this is unexplained, and the obvious reading, that adaptation helps on short paths, is not a mechanism and predicts nothing.

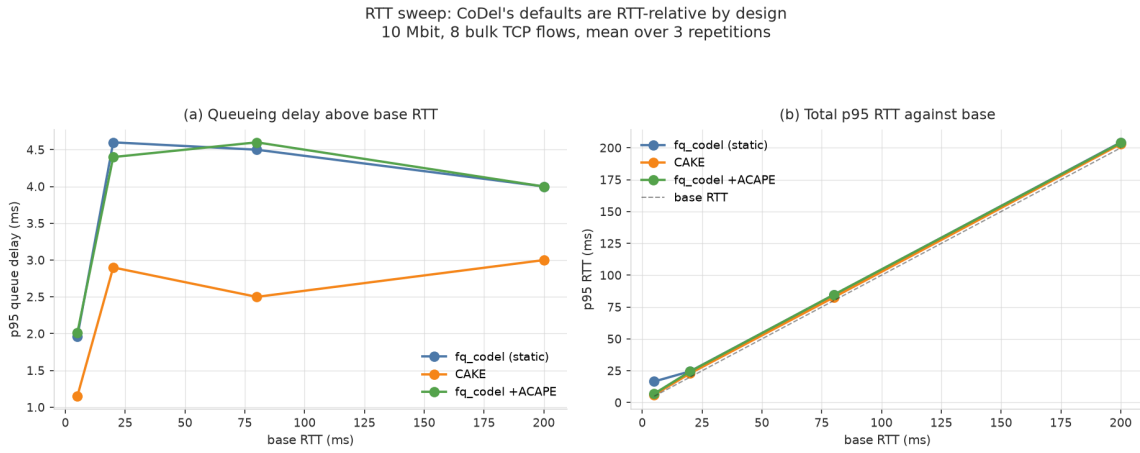


Figure 22: fig15_rtt_sweep.png.

CoDel’s **target** is a delay budget, fixed at 5 ms by default, while the path RTT is whatever the path is. The quantity that should govern whether that budget is badly chosen is therefore neither one alone but their ratio,

$$r = \frac{\text{target}}{\text{RTT}}. \quad (1)$$

On this reading RTT never appears on its own; it appears only through r . That is a stronger claim than the table, and unlike the table it can be wrong.

6.2 Pre-registration

Because a saturating curve can be fitted to almost any monotone data, the hypothesis, its three predictions and their numerical thresholds were committed to the repository *before* the campaign that tests them ran. The commit adding `docs/SCALING_LAW.md` precedes the commit adding the results directory, and the version history shows it.

6.3 Result

$$b(r) = \frac{B}{1 + (r_0/r)^k}, \quad B = 20.5\%, \quad r_0 = 0.50, \quad k = 2.6 \quad (2)$$

with $R^2 = 0.986$. Figure 23 is the evidence. The left panel plots benefit against r , where the measurements collapse onto one curve; filled markers are statistically significant. The right panel plots the identical measurements against path RTT alone, where they do not collapse. The contrast between the panels is the argument.

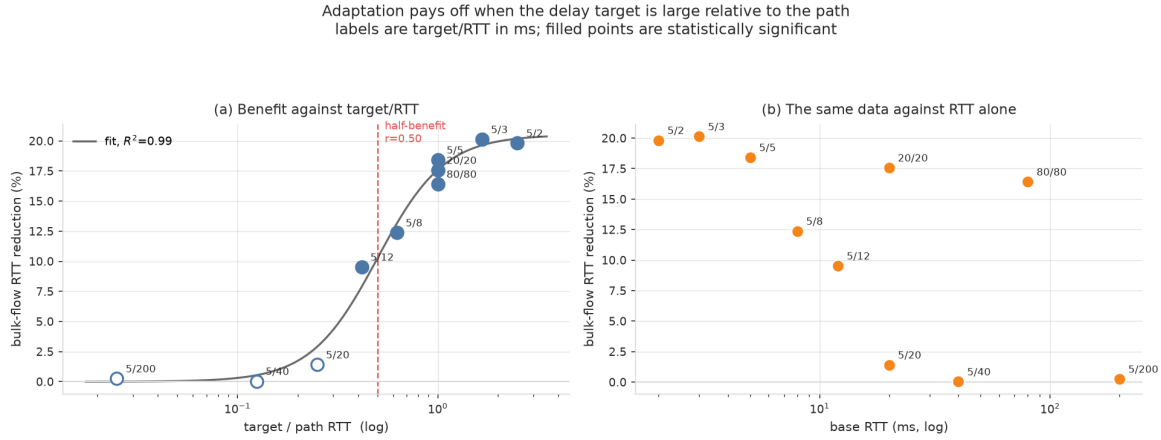


Figure 23: fig16_scaling_law.png.

Table 3: Scaling-law fit and the pre-registered verdicts. Every threshold in the decision column was committed to the repository before the data existed.

target (ms)	RTT (ms)	r	static (ms)	adapted (ms)	reduction (%)	p
5	2	2.500	26.33	21.11	19.8*	0.002
5	3	1.667	26.23	20.95	20.1*	<0.001
5	5	1.000	27.68	22.58	18.4*	<0.001
20	20	1.000	61.36	50.58	17.6*	<0.001
80	80	1.000	181.08	151.35	16.4*	<0.001
5	8	0.625	30.31	26.55	12.4*	<0.001
5	12	0.417	34.18	30.92	9.5*	0.001
5	20	0.250	36.39	35.87	1.4	0.382
5	40	0.125	52.75	52.73	0.0	0.896
5	200	0.025	208.00	207.47	0.3	0.433

Ratio invariance is the prediction able to refute the claim, and it holds. Scaling **target**, **interval**, the control period and the run duration together by the same factor gives 18.4%, 17.6% and 16.4% at 5, 20 and 80 ms, a spread of 2.0 percentage points across a sixteenfold range of RTT. Held-out interior ratios are predicted to 1.4 points. At a fixed 20 ms path, changing only r moves the benefit from 1.4% to 17.6%, which no account in terms of RTT reproduces.

6.4 What it costs, and what it means

The gain is bought with up to 63% more drops and comparable retransmissions, at a throughput cost never exceeding -0.4% in any cell measured. The ratio governs the exchange rate as well as the gain.

Published guidance places **target** at 5 to 10 percent of **interval**, with **interval** on the order of the path RTT, so a correctly configured deployment sits at r between 0.05 and 0.10. Half the available benefit arrives at $r = 0.50$, five to ten times outside that envelope. Parameter adaptation is therefore best understood as a remedy for misconfiguration rather than an improvement on correct configuration, which explains the null results in this work and elsewhere as one effect rather than several.

7 Measurement defects found and corrected

Nine defects were found during this work. Each produced results that were confident, internally consistent and wrong. They are listed because they are the reason the final numbers are believable, and because four of them share one shape.

#	defect	consequence if uncorrected
1	two-node testbed shaped the acknowledgement path	every backlog and drop figure measured ACK queueing
2	eBPF telemetry returned zero while correctly loaded	flow-aware control ran on no information
3	controller state could not reach the kernel	adjustments logged but never applied
4	a control input with no effect on output	identical action reported as though some were predictive
5	constants that looked like measurements	derived values quoted as observations
6	absolute parameter bounds	an 80 ms target clamped to 20 ms on sight, manufacturing a confirmation
7	absolute additive step sizes	controller adapted an order of magnitude more slowly at large targets, manufacturing a refutation
8	absolute regime thresholds	three cells requiring identical treatment received three different control laws
9	<code>iperf3 -logfile</code> appends	re-running a cell corrupted its own goodput and latency silently

Defects 6 to 8, with the control-period defect they concealed, share a single cause: an absolute constant inside a controller whose entire subject is a dimensionless ratio. Two of them would have manufactured a confirmation of the scaling law and two a refutation, and each was invisible until the one before it was fixed. None was visible in a summary statistic; all were found by reading a parameter trajectory or a log file.

8 Threats to validity

Emulation. Experiments run under QEMU without hardware virtualisation. Shaping accuracy was validated before collecting results, and absolute latencies are dominated by configured delay rather than host jitter. Much higher link rates would need revalidation.

Scale. A 10 Mbit bottleneck with 8 to 24 flows represents a home or access link, not a datacentre or backbone.

The second dimensionless group. Holding r at 1.0 and varying only link rate from 2 to 50 Mbit, the benefit is not constant: it peaks near 18% at 5 to 10 Mbit and falls to 13.2% at 2 Mbit and 7.1% at 50 Mbit. A second group, `target/(MTU/rate)`, therefore matters and the single-variable law is incomplete. The prediction that benefit would be suppressed below one packet’s serialisation time was wrong: at 2 Mbit a packet takes 6.06 ms against a 5 ms target, which CoDel cannot reach, yet adaptation still recovers 13.2% by reducing occupancy above that floor.

Cross-mechanism transfer. Fitted independently, plain `code1` places the crossover at $r_0 = 0.42$ against `fq_codel`’s 0.49, and PIE at 1.06. The spread exceeds the factor of two set in advance, and the cause cannot be separated here: PIE’s native `target/tupdate` is 1.0 where CoDel’s `target/interval` is 0.05, so sweeping each discipline at its own defaults varied a second group across the comparison intended to isolate mechanism.

Single bottleneck. Multi-hop topologies with several congested links are not covered.

9 Conclusion

The question of whether adapting a queue discipline’s parameters at runtime improves on its defaults is underspecified, and a single dimensionless number completes it. The benefit is governed by $r = \text{target}/\text{RTT}$, saturating near 20.5% and reaching half of that at $r = 0.50$, invariant across a sixteenfold range of RTT at fixed ratio, and statistically indistinguishable from zero below $r \approx 0.25$ even where the controller demonstrably acts.

Because correct configuration sits an order of magnitude below the crossover, runtime parameter adaptation is a remedy for misconfiguration rather than an improvement on correct configuration. That is narrower than the systems literature claims and more useful than the evaluations reporting nothing, and it accounts for both.

A Complete figure set

Figures already discussed above are not repeated. The remainder of the analysis output follows, with captions from the figure index.

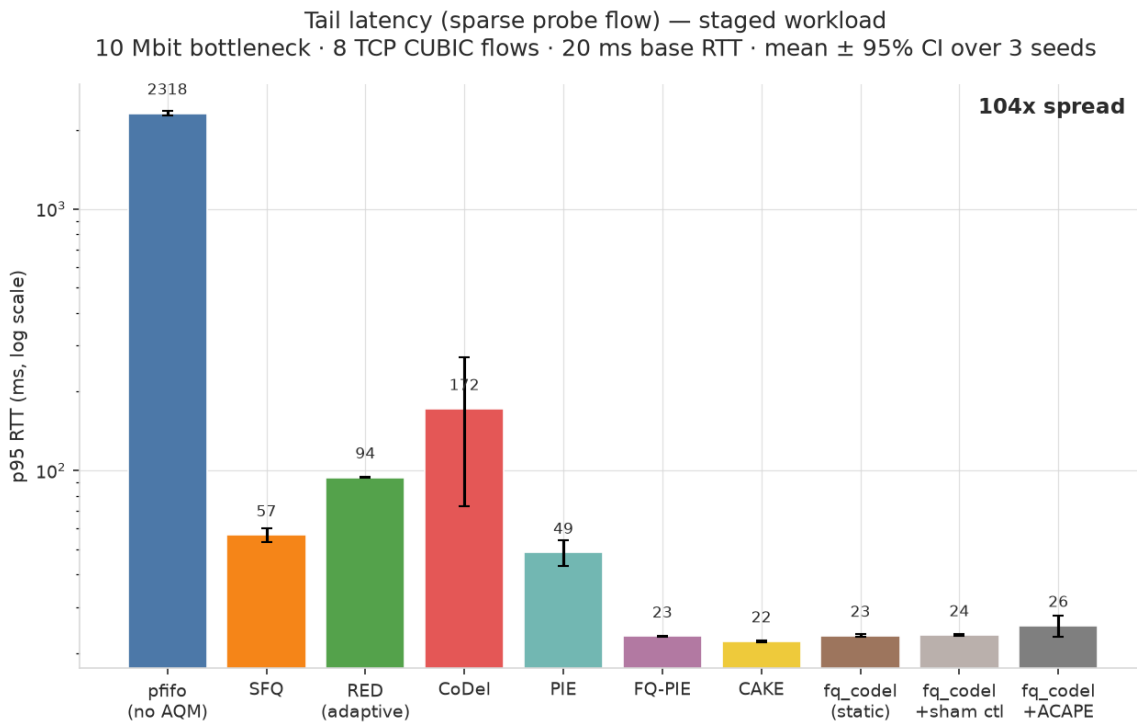


Figure 24: fig01_latency_tail_staged.png. Tail latency (p95 RTT, sparse probe flow) across all systems, log scale (staged workload)

B Implementation captures not shown above

C Per-run figures

One figure per experimental run, 63 in all, each showing that run’s time series. They are tabulated rather than embedded because the analysis figures above aggregate them; every file is in `figures/runs/`.

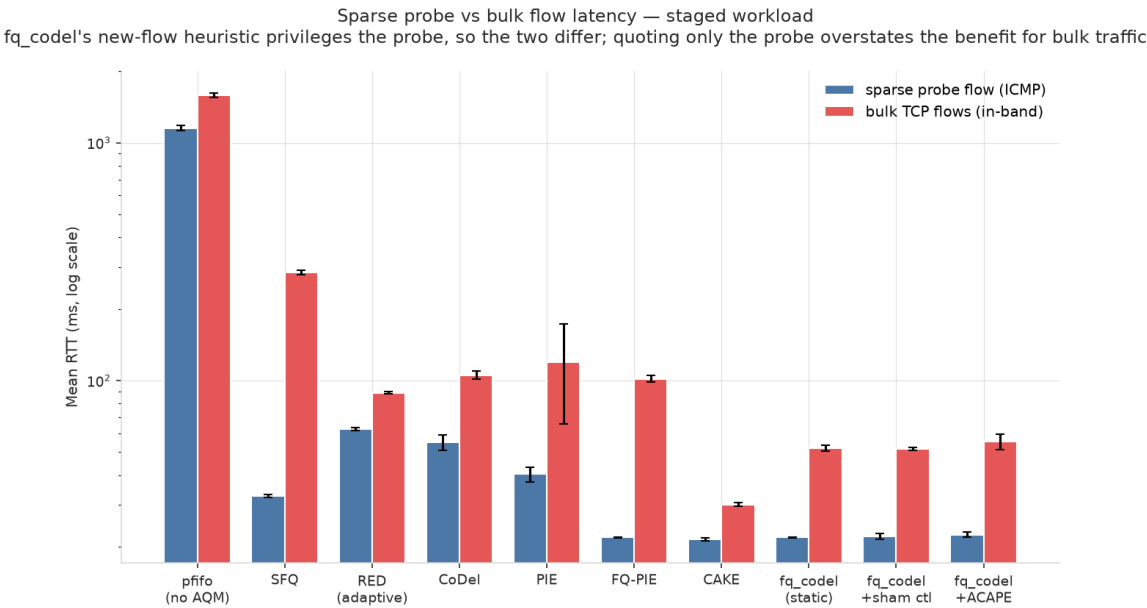


Figure 25: fig02_latency_sparse_vs_bulk_staged.png. Sparse probe flow vs bulk TCP flow RTT, why the two differ (staged workload)

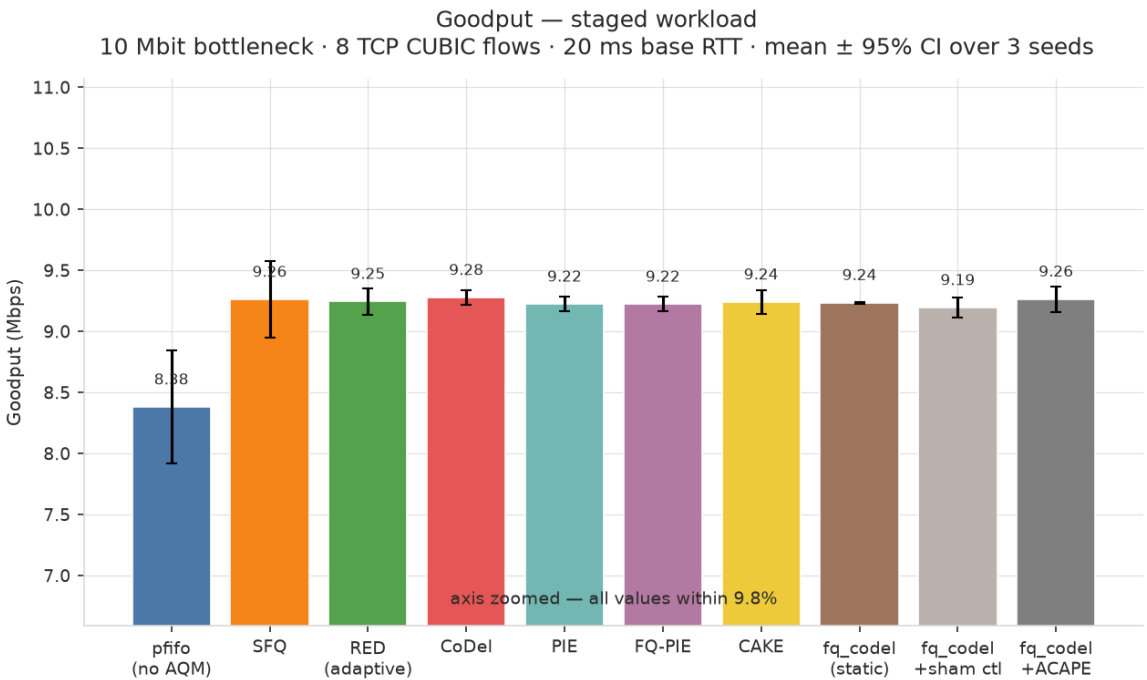


Figure 26: fig03_throughput_staged.png. Goodput (from iperf3 sum_received), axis zoomed to show it is flat (staged workload)

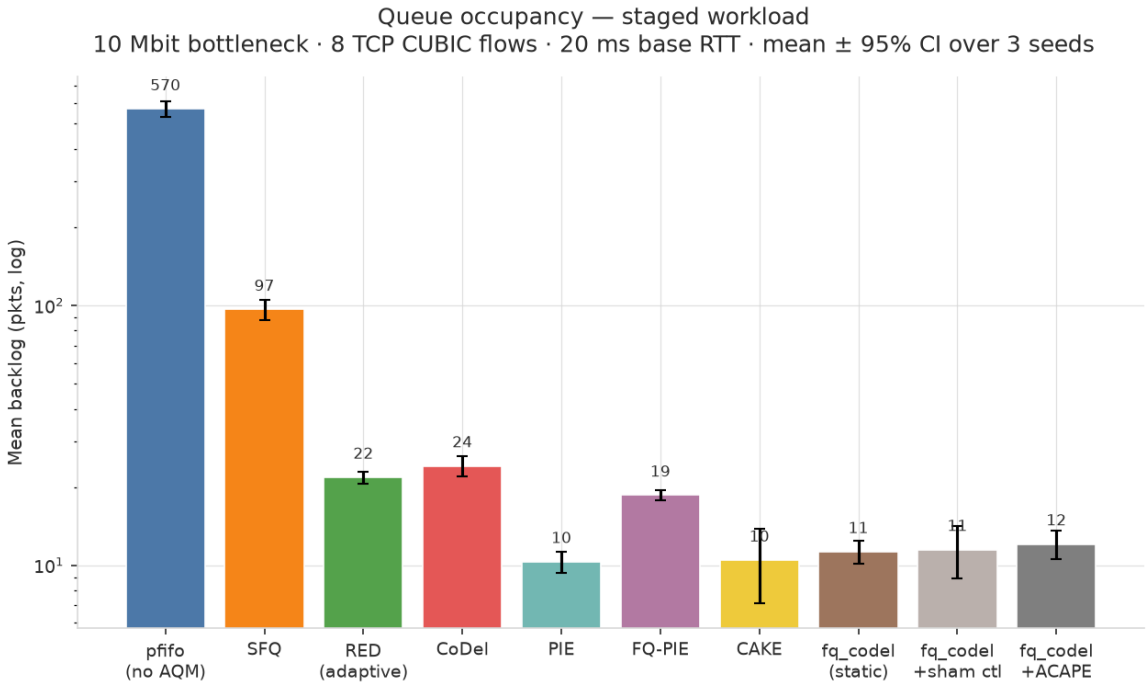


Figure 27: fig04_backlog_staged.png. Mean queue occupancy, log scale (staged workload)

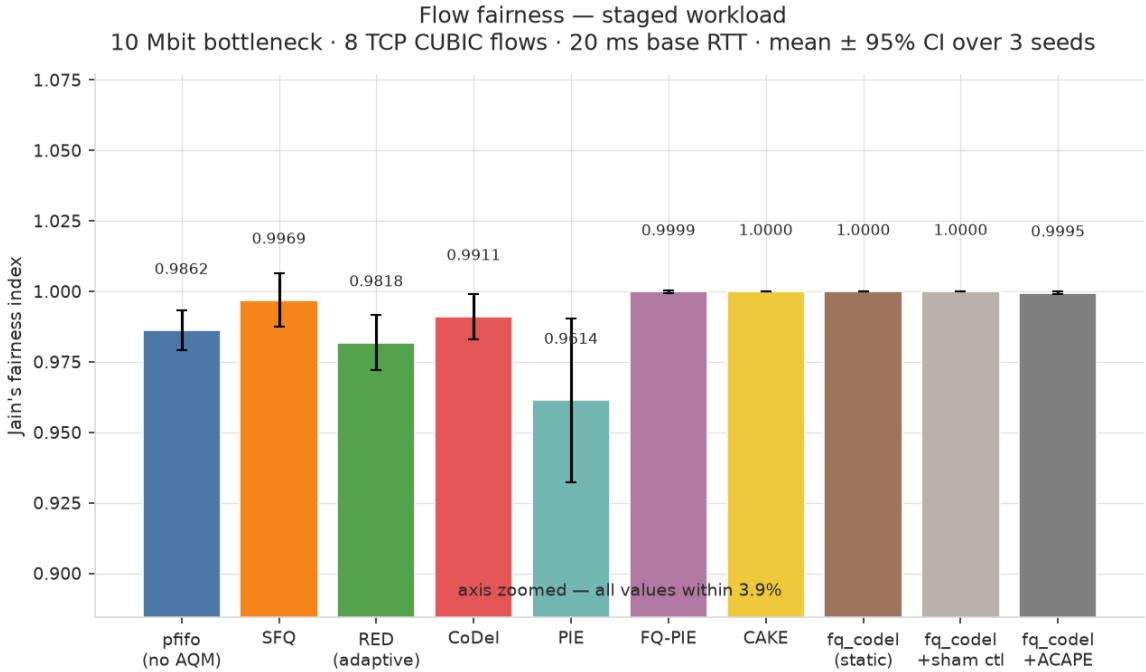


Figure 28: fig05_fairness_staged.png. Jain's fairness index, axis zoomed (staged workload)

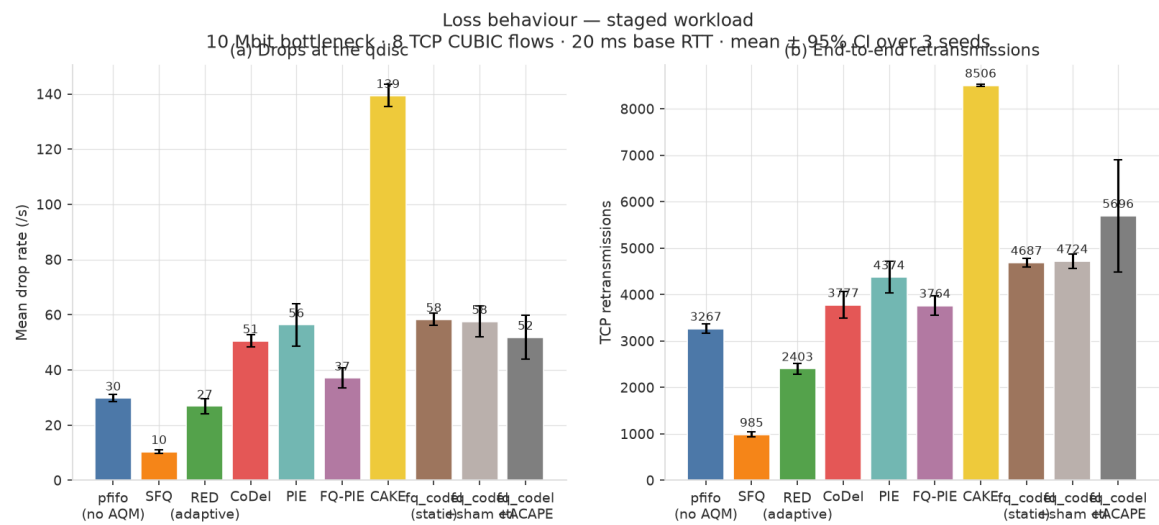


Figure 29: fig06_droprate_staged.png. AQM drop rate and end-to-end TCP retransmissions (staged workload)

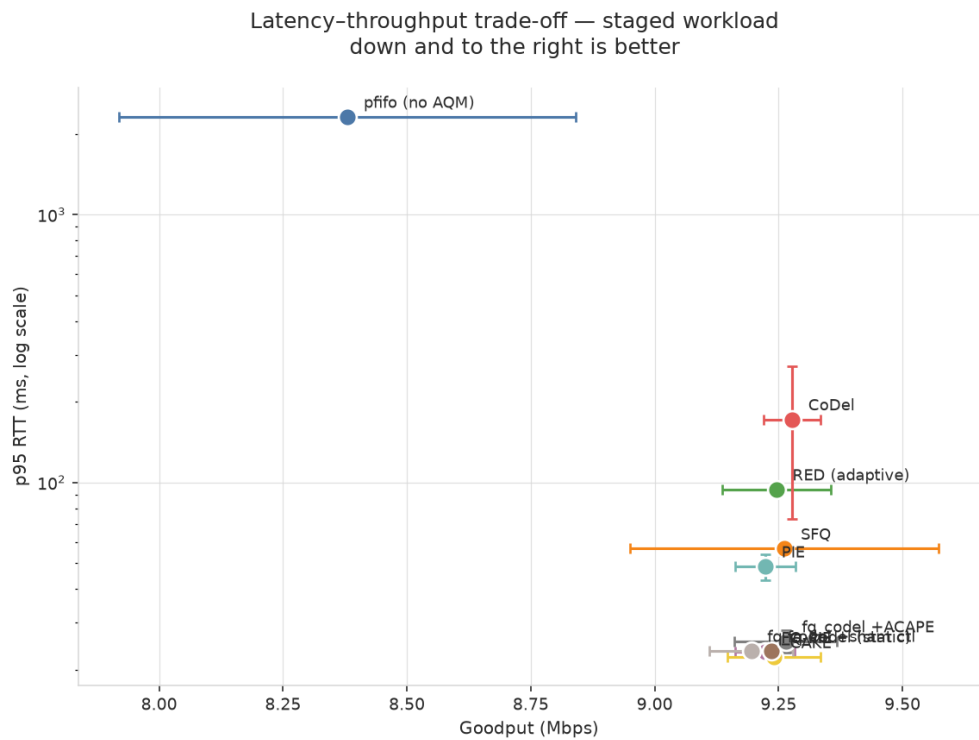


Figure 30: fig07_tradeoff_staged.png. Latency vs throughput trade-off (staged workload)

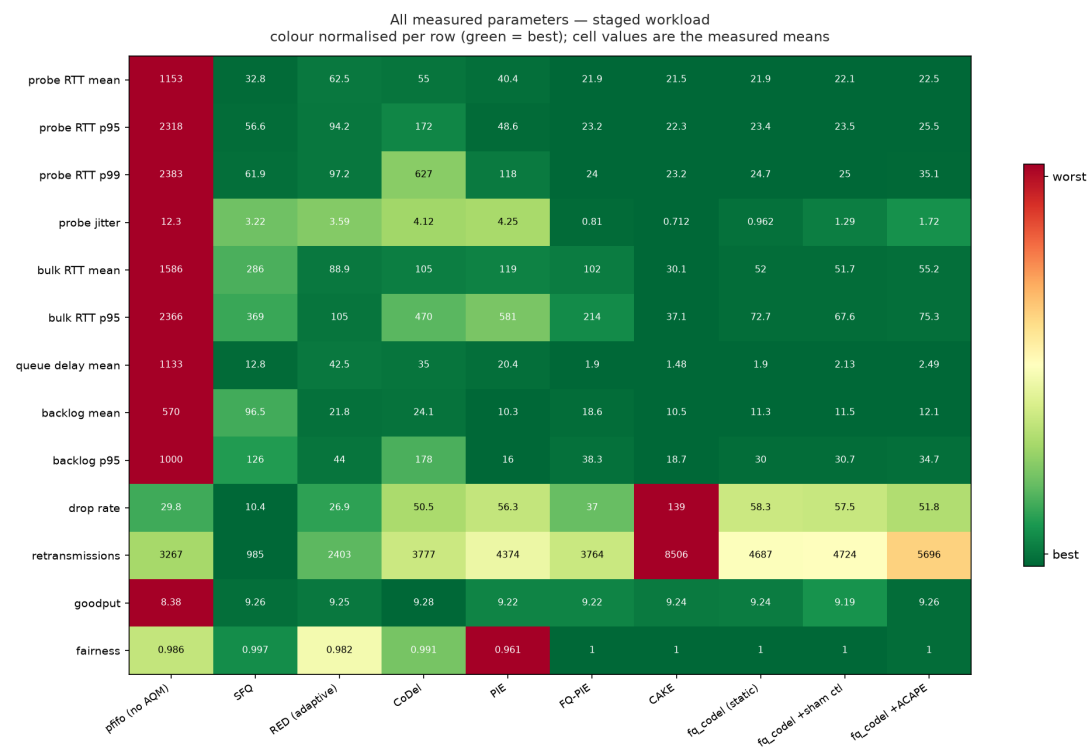


Figure 31: fig08_allparams_staged.png. All measured parameters × all systems, normalised heatmap with measured values (staged workload)

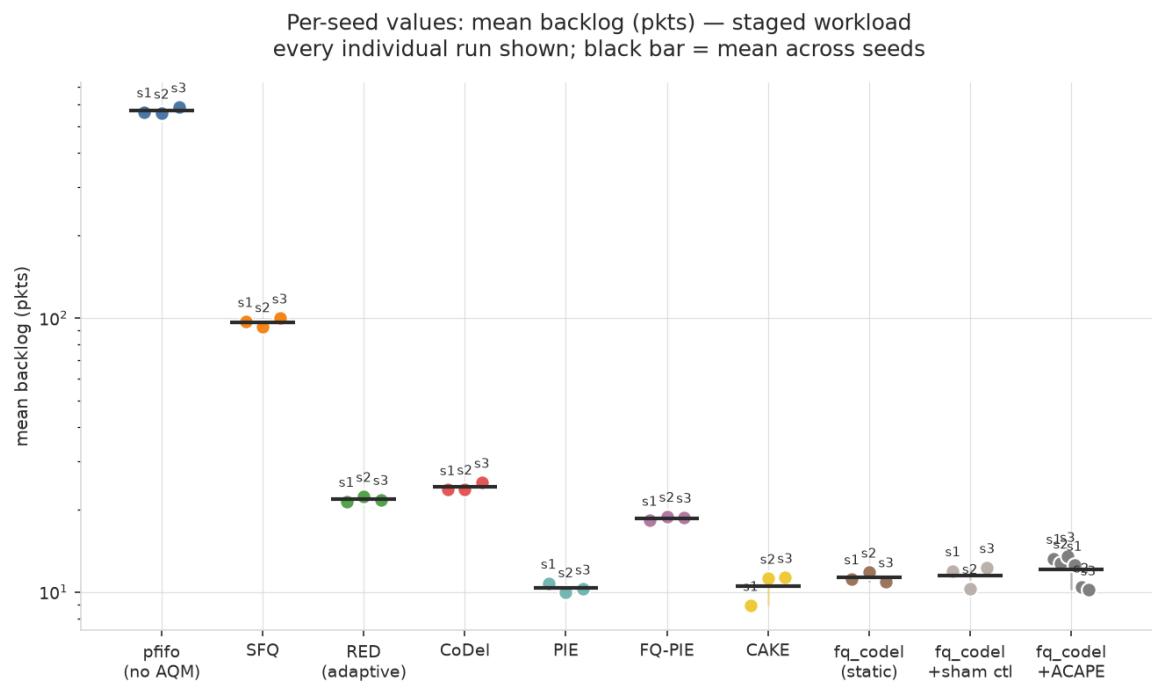


Figure 32: fig09_seeds_backlog_mean_pkts_staged.png. Per-seed values, every individual run shown, mean marked, backlog mean pkts (staged workload)

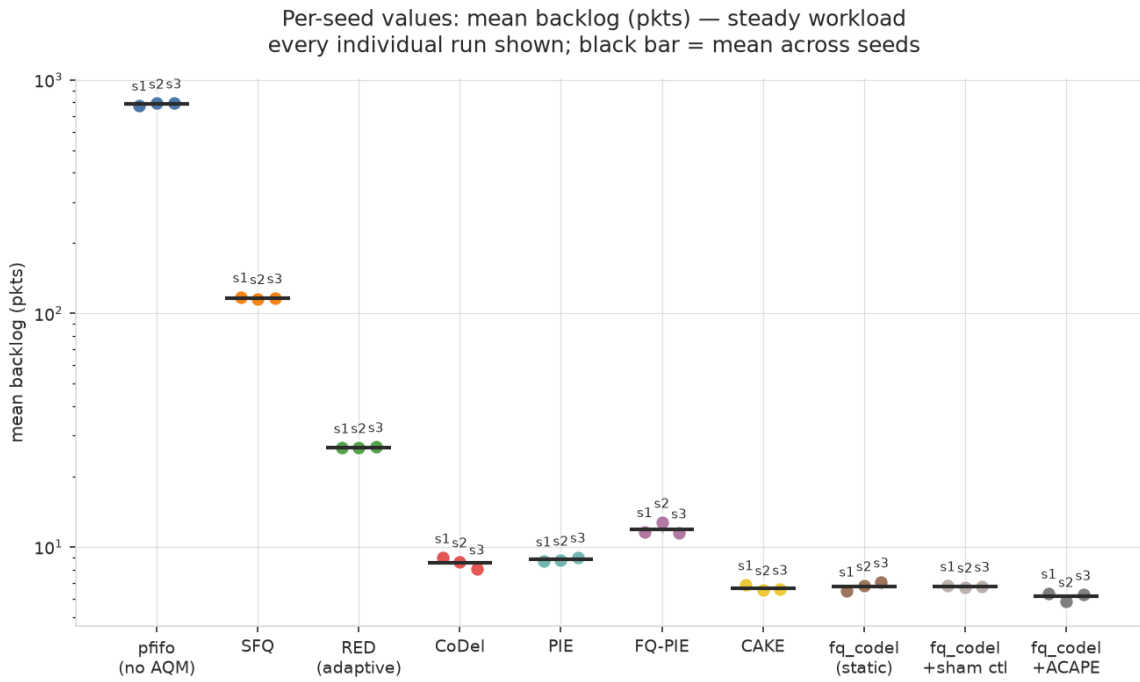


Figure 33: fig09_seeds_backlog_mean_pkts_steady.png. Per-seed values, every individual run shown, mean marked, backlog mean pkts (steady workload)

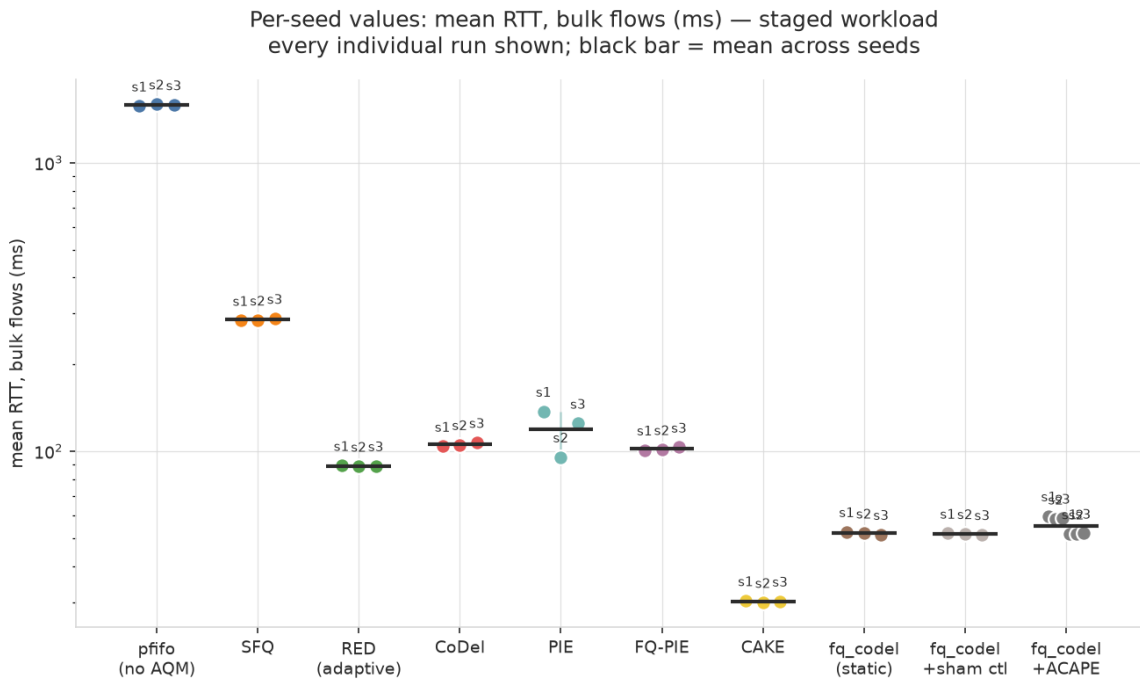


Figure 34: fig09_seeds_bulk_rtt_mean_ms_staged.png. Per-seed values, every individual run shown, mean marked, bulk rtt mean ms (staged workload)

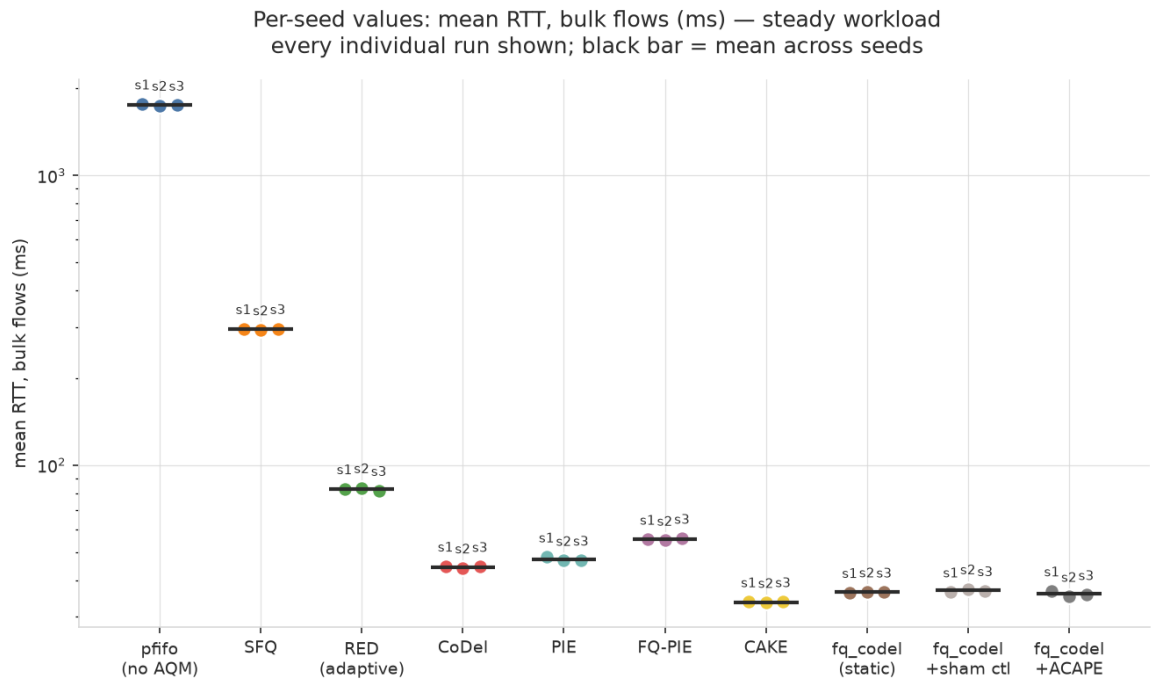


Figure 35: fig09_seeds_bulk_rtt_mean_ms_steady.png. Per-seed values, every individual run shown, mean marked, bulk rtt mean ms (steady workload)

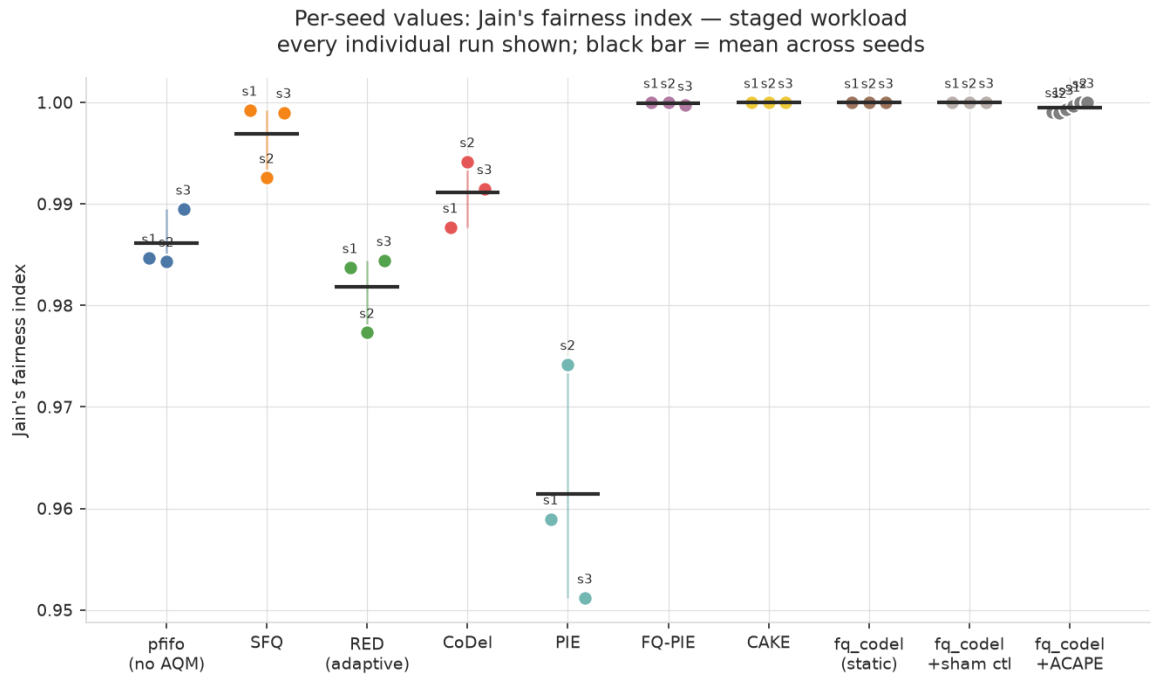


Figure 36: fig09_seeds_jain_staged.png. Per-seed values, every individual run shown, mean marked, jain (staged workload)

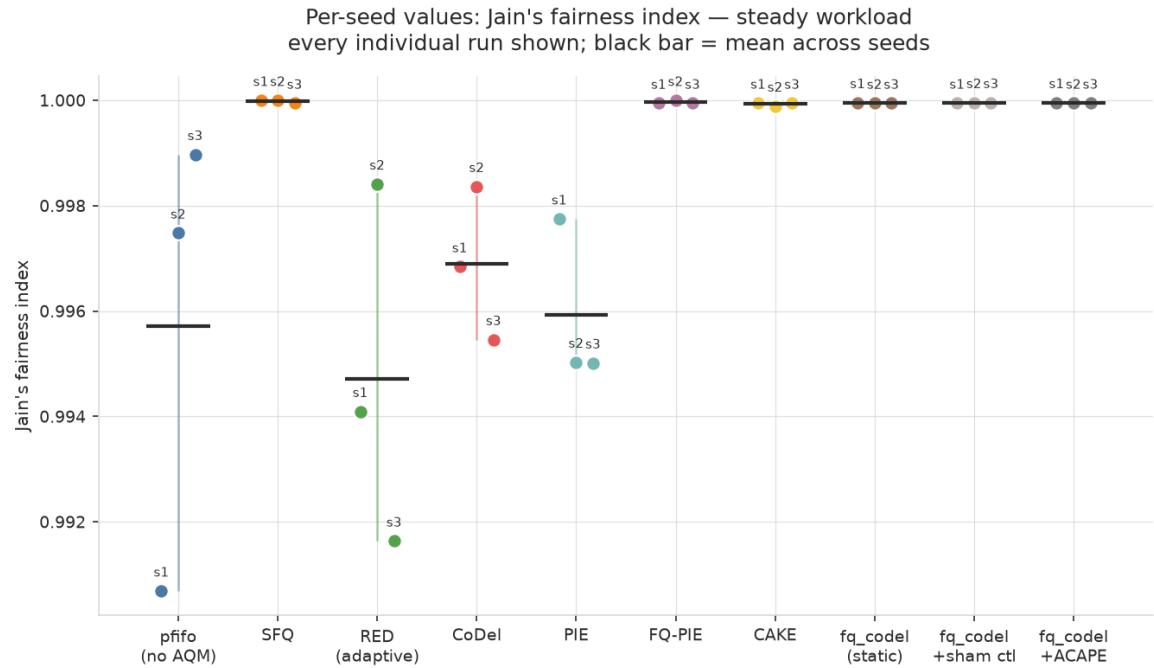


Figure 37: fig09_seeds_jain_steady.png. Per-seed values, every individual run shown, mean marked, jain (steady workload)

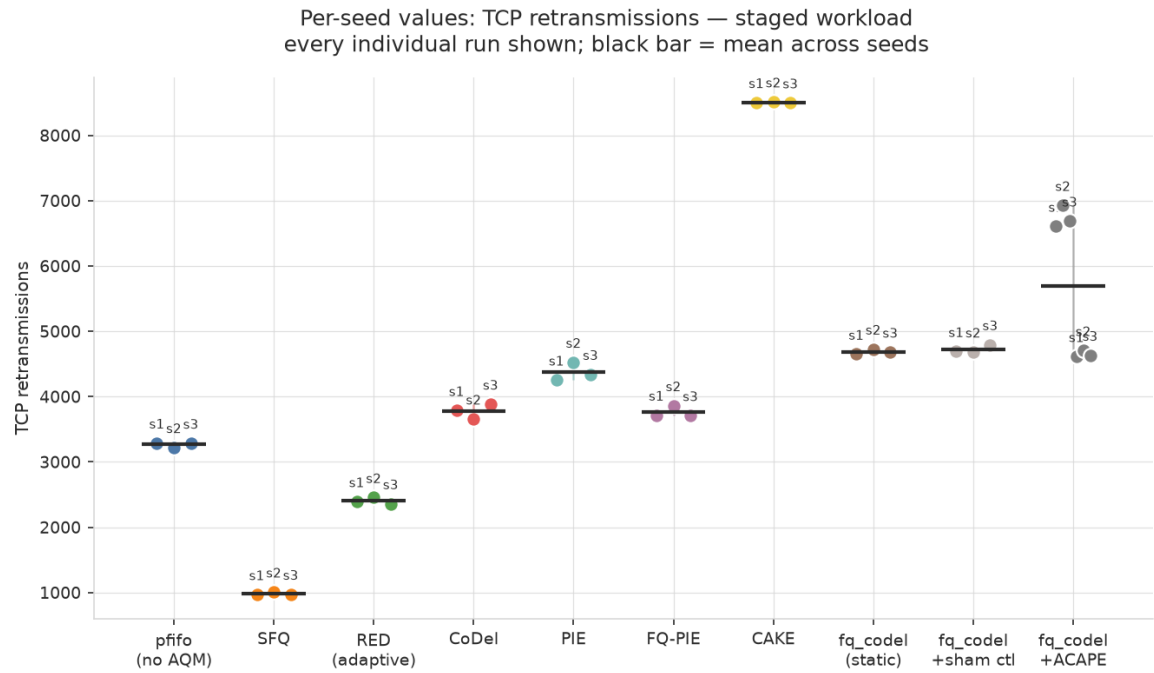


Figure 38: fig09_seeds_retransmits_staged.png. Per-seed values, every individual run shown, mean marked, retransmits (staged workload)

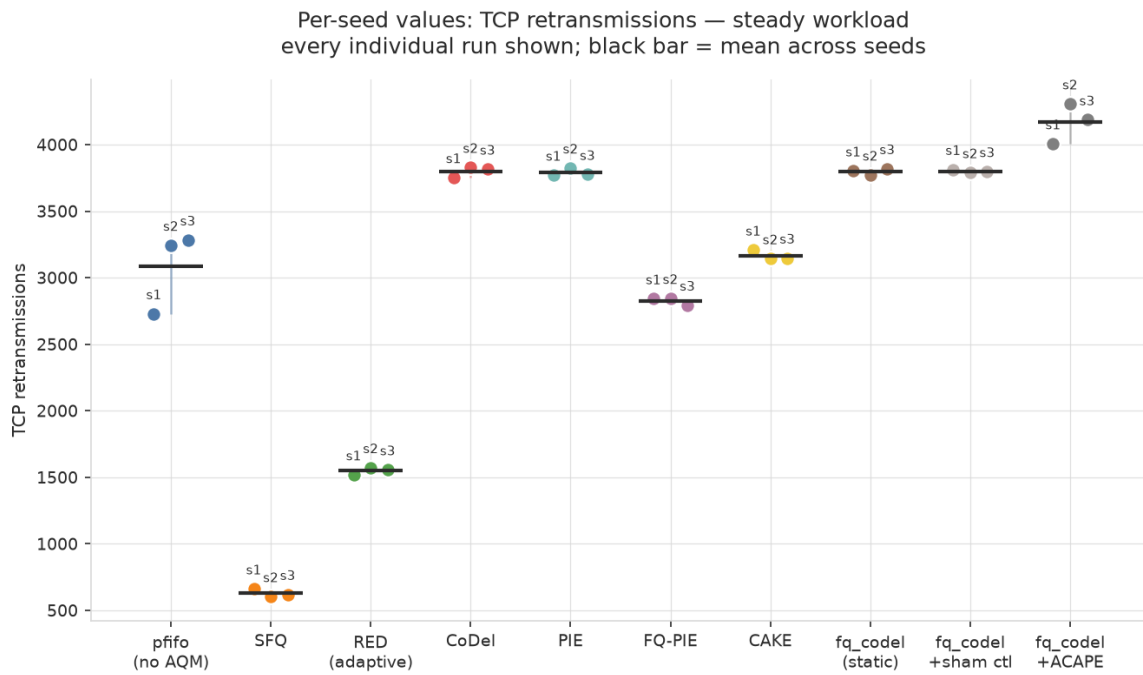


Figure 39: fig09_seeds_retransmits_steady.png. Per-seed values, every individual run shown, mean marked, retransmits (steady workload)

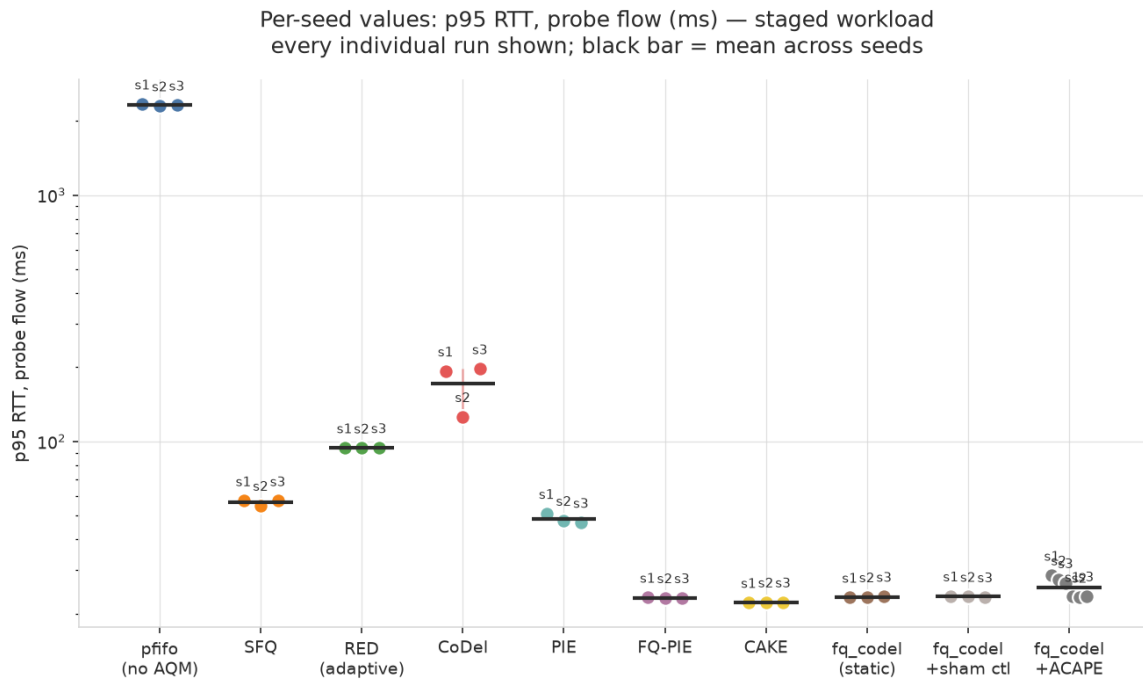


Figure 40: fig09_seeds_sparse_rtt_p95_ms_staged.png. Per-seed values, every individual run shown, mean marked, sparse rtt p95 ms (staged workload)

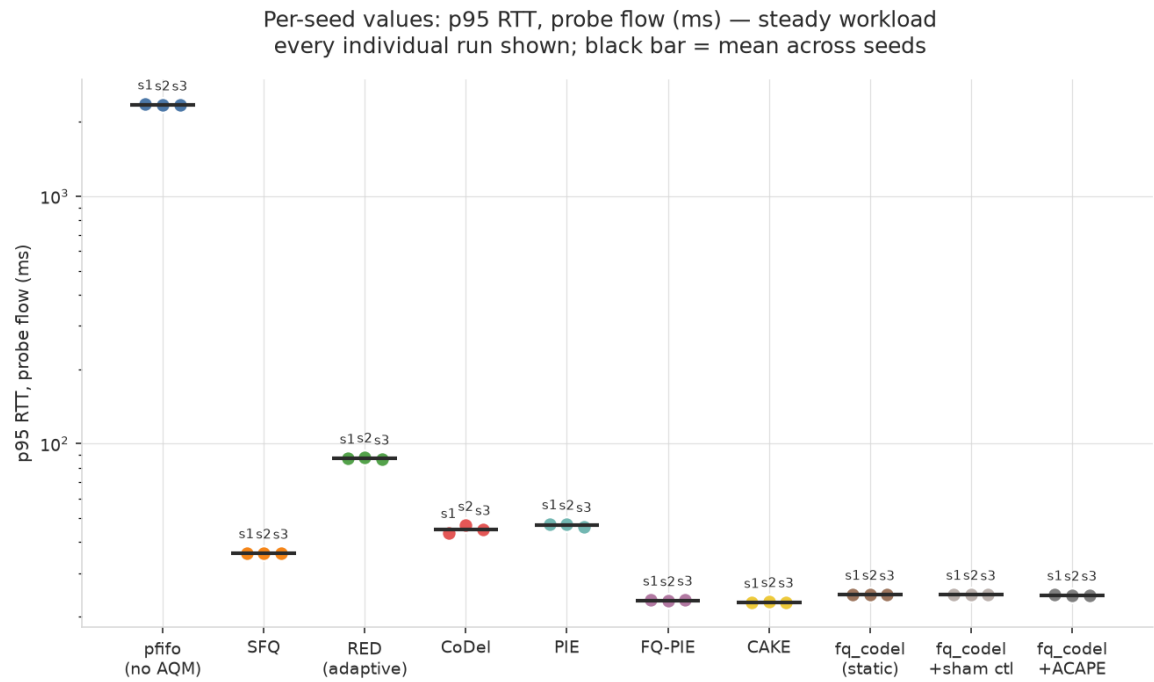


Figure 41: fig09_seeds_sparse_rtt_p95_ms_steady.png. Per-seed values, every individual run shown, mean marked, sparse rtt p95 ms (steady workload)

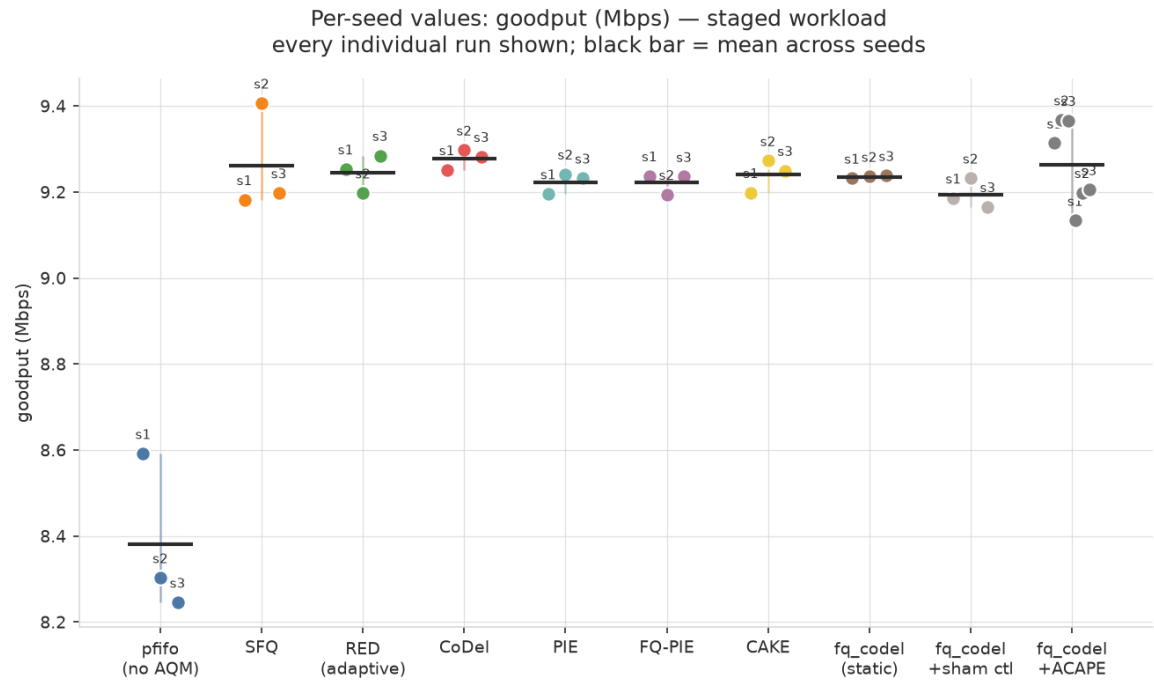


Figure 42: fig09_seeds_throughput_mbps_staged.png. Per-seed values, every individual run shown, mean marked, throughput mbps (staged workload)

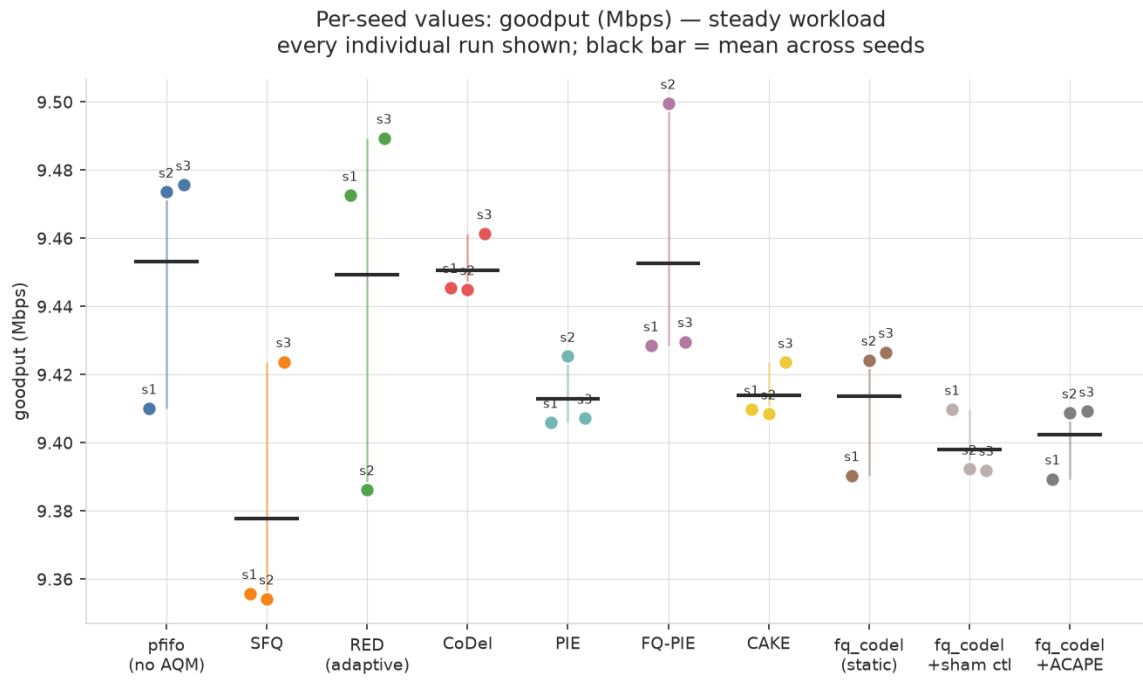


Figure 43: fig09_seeds_throughput_mbps_steady.png. Per-seed values, every individual run shown, mean marked, throughput mbps (steady workload)



Figure 44: fig10_timeseries_backlog_steady.png. Queue occupancy over time (steady workload)



Figure 45: fig11_timeseries_rtt_steady.png. Measured RTT over time (20 Hz probe) (steady workload)

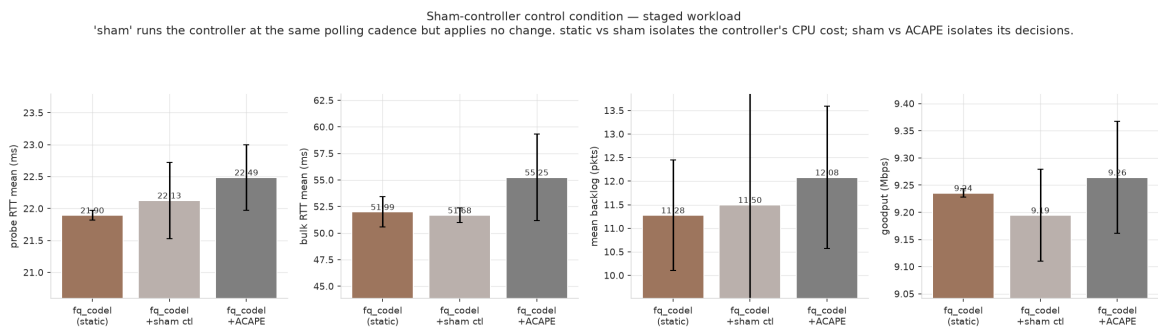


Figure 46: fig13_sham_control_staged.png. Sham-controller condition, separates controller CPU cost from control decisions (staged workload)

```

STEP 09 Why the eBPF telemetry read zero in every historical run
Three independent defects in read_flows(), each fatal on its own. The immediate cause was a
TypeError swallowed by a bare except; behind it sat wrong struct offsets and a comparison of
CLOCK_REALTIME against CLOCK_MONOTONIC giving ages of decades.

$ cd /home/user/ccn-linux-qdisc-study && python3 verification/demo_bugs.py

=====
BUG 1 bpftool --json yields hex STRINGS; bytes() on them raises
=====
entry['value'][:4] = ['0xa0', '0xb8', '0x00', '0x00']
bytes(raw[16:24]) -> TypeError: 'str' object cannot be interpreted as an integer
acape_v5.py wrapped this in 'except: pass', so read_flows() returned
zeros on every tick and the failure was never visible.

=====
BUG 2 struct offsets omitted first_seen_ns, shifting every later field
=====
true : packets@0 bytes@8 first_seen@16 last_seen@24 gap@32 elephant@40
v5   : packets@0 bytes@8 last_ns@16 gap@24 elephant@32

offset 16 = 502568911582 truly first_seen_ns, v5 read it as last_ns
offset 24 = 512572233902 truly last_seen_ns, v5 read it as gap_ns
offset 32 = 181126 truly gap_ns; v5 read its low 4 bytes as
           0 is_elephant -> 181126
offset 40 = 0 the real is_elephant flag

=====
BUG 3 CLOCK_REALTIME compared against CLOCK_MONOTONIC
=====
time.time_ns() = 1798695534927932582 (since 1970)
bpf ktime_get_ns() = 512572233902 (since boot)
v5 age = (realtime - ktime)/1e9 = 1,790,695,022 s = 56.8 YEARS
filter 'if age < 2.0' -> False

corrected comparison uses CLOCK_MONOTONIC on both sides -> age ~ 0.0010s, passes the filter

=====
RESULT: active_flows = 0 in 21,128 of 21,128 recorded ticks (100%).
The paper attributed this to 'namespace file-descriptor isolation'.
The maps were populated throughout; the fault was entirely in userspace.
=====

captured 2026-09-29 15:25:34 UTC exit=0 kernel 6.18.44-rc-v49

```

Figure 47: Why the eBPF telemetry read zero in every historical run. Three independent defects in `read_flows()`, each fatal on its own. The immediate cause was a `TypeError` swallowed by a bare `except`; behind it sat wrong struct offsets and a comparison of `CLOCK_REALTIME` against `CLOCK_MONOTONIC` giving ages of decades.

```

STEP 10 Throughput overshoot: sum_sent exceeds the link rate
iperf3's sum_sent counts bytes handed to the socket, not bytes delivered. Under pfifo's 1000-packet
buffer the sender fills the queue and reports 12.16 Mbps on a 10 Mbit link - physically impossible
as a throughput. The excess appears as an 827-packet backlog and 2381 ms p95 RTT, not as delivered
data. Every flow-queueing AQM shows no overshoot. This is why goodput is taken from sum_received.

$ cd /home/user/ccn-linux-qdisc-study && python3 verification/demo_overshoot.py results

=====
THROUGHPUT OVERSHOOT: iperf3 sum_sent vs sum_received on a 10 Mbit link
=====
run      link      sum_sent  sum_recv  overshoot  backlog  p95 RTT
-----
cake_steady_s1  10      9.47      9.41      -0.53      6.9      22.8
cake_steady_s2  10      9.50      9.41      -0.50      6.5      23.0
cake_steady_s3  10      9.50      9.42      -0.50      6.6      22.9
codel_steady_s1 10      9.73      9.45      -0.27      9.0      43.7
codel_steady_s2 10      9.71      9.45      -0.29      8.6      46.7
codel_steady_s3 10      9.73      9.46      -0.27      8.0      44.9
fq_codel_acape_steady_s1 10  9.55      9.39      -0.45      6.3      24.5
fq_codel_acape_steady_s2 10  9.59      9.41      -0.41      5.8      24.4
fq_codel_acape_steady_s3 10  9.61      9.41      -0.39      6.2      24.4
fq_codel_sham_steady_s1 10  9.61      9.41      -0.39      6.8      24.6
fq_codel_sham_steady_s2 10  9.59      9.39      -0.41      6.7      24.6
fq_codel_sham_steady_s3 10  9.57      9.39      -0.43      6.8      24.5
fq_codel_steady_s1 10  9.55      9.39      -0.45      6.5      24.6
fq_codel_steady_s2 10  9.60      9.42      -0.40      6.8      24.6
fq_codel_steady_s3 10  9.57      9.43      -0.43      7.1      24.5
fq_pie_steady_s1 10  9.75      9.43      -0.25      11.6     23.3
fq_pie_steady_s2 10  9.78      9.50      -0.22      12.7     23.2
fq_pie_steady_s3 10  9.75      9.43      -0.25      11.5     23.3
prifo_steady_s1 10  12.03     9.41      +2.03      778.6    2359.0 <== IMPOSSIBLE
prifo_steady_s2 10  11.91     9.47      +1.91      793.8    2339.0 <== IMPOSSIBLE
prifo_steady_s3 10  12.16     9.48      +2.16      795.3    2334.0 <== IMPOSSIBLE
pie_steady_s1 10  9.69      9.41      -0.31      8.7      47.4
pie_steady_s2 10  9.71      9.43      -0.29      8.8      47.2
pie_steady_s3 10  9.69      9.41      -0.31      9.1      46.2
red_steady_s1 10  9.68      9.47      -0.32      26.6     87.3
red_steady_s2 10  9.59      9.39      -0.41      26.6     88.3
red_steady_s3 10  9.68      9.49      -0.32      26.9     86.5
sfq_steady_s1 10  9.80      9.36      -0.20      117.6    36.2
sfq_steady_s2 10  9.78      9.35      -0.22      115.5    36.1
sfq_steady_s3 10  9.81      9.42      -0.19      116.4    36.0

=====
WHY
=====
A 10 Mbit link cannot deliver more than 10 Mbps. Any 'throughput'
figure above the link rate is measuring what the SENDER queued, not
what the receiver got.

prifo holds ~1000 packets, so the sender fills the buffer and iperf3
counts those bytes as sent. The excess shows up as queueing delay,
... [7 more lines omitted]

captured 2026-09-29 15:25:35 UTC exit=0 kernel 6.18.44-rc-v49

```

Figure 48: Throughput overshoot: `sum_sent` exceeds the link rate. `iperf3`'s `sum_sent` counts bytes handed to the socket, not bytes delivered. Under `prifo`'s 1000-packet buffer the sender fills the queue and reports 12.16 Mbps on a 10 Mbit link - physically impossible as a throughput. The excess appears as an 827-packet backlog and 2381 ms p95 RTT, not as delivered data. Every flow-queueing AQM shows no overshoot. This is why goodput is taken from `sum_received`.

```

STEP 11 Drop rates in the original logs exceed the link's packet rate
A 10 Mbit link carrying 1514-byte packets passes at most 826 packets/s, and iperf3 reported ~179
retransmits/s. The original logs record 100,000-138,000 drops/s - up to 155x the physical packet
rate. This is the arithmetic that first showed the measurements could not describe the data path.

$ cd /home/user/ccn-linux-qdisc-study && python3 verification/demo_droprate.py

=====
DROP RATES IN THE ORIGINAL LOGS vs THE LINK'S PHYSICAL PACKET RATE
=====

Link rate           : 10 Mbit/s
Full-size packet    : 1514 bytes
Maximum packets/s   : 826
iperf3 reported      : ~179 retransmits/s

log                  median drop/s   max drop/s   x link rate
-----
acape_metrics_20260328_155133.csv      128208        138407       155x
acape_metrics_20260328_155950.csv      127067        137445       154x
acape_metrics_20260401_112753.csv      123957        138379       150x
acape_metrics_20260331_113641.csv      102638        128407       124x
acape_metrics_20260402_105315.csv      101851        132956       123x
acape_metrics_20260328_154521.csv       62745        138211        76x
acape_metrics_20260328_074014.csv       53368        133394        65x
acape_metrics_20260402_131706.csv       50594        65866         61x
acape_metrics_20260328_172221.csv       49173        76601         60x
acape_metrics_20260328_144155.csv       22781        134220        28x
acape_metrics_20260328_091002.csv       22212        131112        27x
acape_metrics_20260328_090222.csv       19932        135363        24x
acape_metrics_20260330_004323.csv       10492        134950        13x
acape_metrics_20260328_071737.csv        9982        107135        12x

15 of 32 runs have a MEDIAN drop rate above the physical packet rate.

A queue cannot drop more packets per second than can arrive. These
figures are consistent with a stream of 66-byte ACKs overflowing the
shaper (10e6/(66*8) = 18,939 ACK/s admitted), not with data queueing.
=====

captured 2026-09-29 15:25:35 UTC  exit=0  kernel 6.18.44-fc-v49

```

Figure 49: Drop rates in the original logs exceed the link's packet rate. A 10 Mbit link carrying 1514-byte packets passes at most 826 packets/s, and iperf3 reported ~179 retransmits/s. The original logs record 100,000-138,000 drops/s - up to 155x the physical packet rate. This is the arithmetic that first showed the measurements could not describe the data path.

ID	run	summary
RUN 01	cake_staged_s1	cake, staged workload, seed 1, goodput 9.1985 Mbps, p95 RTT 22.2 ms, backlog 8.92 pkt
RUN 02	cake_staged_s2	cake, staged workload, seed 2, goodput 9.2728 Mbps, p95 RTT 22.3 ms, backlog 11.2 pkt
RUN 03	cake_staged_s3	cake, staged workload, seed 3, goodput 9.25 Mbps, p95 RTT 22.3 ms, backlog 11.28 pkt
RUN 04	cake_steady_s1	cake, steady workload, seed 1, goodput 9.4098 Mbps, p95 RTT 22.8 ms, backlog 6.89 pkt
RUN 05	cake_steady_s2	cake, steady workload, seed 2, goodput 9.4085 Mbps, p95 RTT 23.0 ms, backlog 6.55 pkt
RUN 06	cake_steady_s3	cake, steady workload, seed 3, goodput 9.4236 Mbps, p95 RTT 22.9 ms, backlog 6.58 pkt
RUN 07	codell_staged_s1	codell, staged workload, seed 1, goodput 9.2517 Mbps, p95 RTT 193.0 ms, backlog 23.58 pkt
RUN 08	codell_staged_s2	codell, staged workload, seed 2, goodput 9.2975 Mbps, p95 RTT 126.0 ms, backlog 23.71 pkt
RUN 09	codell_staged_s3	codell, staged workload, seed 3, goodput 9.2825 Mbps, p95 RTT 197.0 ms, backlog 25.09 pkt
RUN 10	codell_steady_s1	codell, steady workload, seed 1, goodput 9.4456 Mbps, p95 RTT 43.7 ms, backlog 8.98 pkt
RUN 11	codell_steady_s2	codell, steady workload, seed 2, goodput 9.445 Mbps, p95 RTT 46.7 ms, backlog 8.63 pkt
RUN 12	codell_steady_s3	codell, steady workload, seed 3, goodput 9.4614 Mbps, p95 RTT 44.9 ms, backlog 8.04 pkt

ID	run	summary
RUN 13	fq_codel_acape_ebpf_staged_s1	fq_codel + ACAPE + eBPF, staged workload, seed 1, goodput None Mbps, p95 RTT 24.7 ms, backlog 11.19 pkt
RUN 14	fq_codel_acape_ebpf_staged_s2	fq_codel + ACAPE + eBPF, staged workload, seed 2, goodput None Mbps, p95 RTT 25.3 ms, backlog 11.93 pkt
RUN 15	fq_codel_acape_ebpf_staged_s3	fq_codel + ACAPE + eBPF, staged workload, seed 3, goodput None Mbps, p95 RTT 24.5 ms, backlog 12.96 pkt
RUN 16	fq_codel_acape_staged_s1	fq_codel + ACAPE, staged workload, seed 1, goodput 9.1339 Mbps, p95 RTT 23.5 ms, backlog 12.5 pkt
RUN 17	fq_codel_acape_staged_s2	fq_codel + ACAPE, staged workload, seed 2, goodput 9.1986 Mbps, p95 RTT 23.3 ms, backlog 10.4 pkt
RUN 18	fq_codel_acape_staged_s3	fq_codel + ACAPE, staged workload, seed 3, goodput 9.2051 Mbps, p95 RTT 23.6 ms, backlog 10.16 pkt
RUN 19	fq_codel_acape_steady_s1	fq_codel + ACAPE, steady workload, seed 1, goodput 9.3894 Mbps, p95 RTT 24.5 ms, backlog 6.34 pkt
RUN 20	fq_codel_acape_steady_s2	fq_codel + ACAPE, steady workload, seed 2, goodput 9.4088 Mbps, p95 RTT 24.4 ms, backlog 5.84 pkt
RUN 21	fq_codel_acape_steady_s3	fq_codel + ACAPE, steady workload, seed 3, goodput 9.4093 Mbps, p95 RTT 24.4 ms, backlog 6.24 pkt
RUN 22	fq_codel_sham_staged_s1	fq_codel, staged workload, seed 1, goodput 9.1858 Mbps, p95 RTT 23.6 ms, backlog 11.93 pkt
RUN 23	fq_codel_sham_staged_s2	fq_codel, staged workload, seed 2, goodput 9.2323 Mbps, p95 RTT 23.5 ms, backlog 10.3 pkt
RUN 24	fq_codel_sham_staged_s3	fq_codel, staged workload, seed 3, goodput 9.1657 Mbps, p95 RTT 23.4 ms, backlog 12.26 pkt
RUN 25	fq_codel_sham_steady_s1	fq_codel, steady workload, seed 1, goodput 9.4098 Mbps, p95 RTT 24.6 ms, backlog 6.82 pkt
RUN 26	fq_codel_sham_steady_s2	fq_codel, steady workload, seed 2, goodput 9.3923 Mbps, p95 RTT 24.6 ms, backlog 6.73 pkt
RUN 27	fq_codel_sham_steady_s3	fq_codel, steady workload, seed 3, goodput 9.392 Mbps, p95 RTT 24.5 ms, backlog 6.79 pkt
RUN 28	fq_codel_staged_s1	fq_codel, staged workload, seed 1, goodput 9.232 Mbps, p95 RTT 23.3 ms, backlog 11.15 pkt
RUN 29	fq_codel_staged_s2	fq_codel, staged workload, seed 2, goodput 9.2362 Mbps, p95 RTT 23.4 ms, backlog 11.8 pkt
RUN 30	fq_codel_staged_s3	fq_codel, staged workload, seed 3, goodput 9.2382 Mbps, p95 RTT 23.5 ms, backlog 10.88 pkt
RUN 31	fq_codel_steady_s1	fq_codel, steady workload, seed 1, goodput 9.3904 Mbps, p95 RTT 24.6 ms, backlog 6.47 pkt
RUN 32	fq_codel_steady_s2	fq_codel, steady workload, seed 2, goodput 9.4243 Mbps, p95 RTT 24.6 ms, backlog 6.83 pkt
RUN 33	fq_codel_steady_s3	fq_codel, steady workload, seed 3, goodput 9.4266 Mbps, p95 RTT 24.5 ms, backlog 7.07 pkt
RUN 34	fq_pie_staged_s1	fq_pie, staged workload, seed 1, goodput 9.2375 Mbps, p95 RTT 23.3 ms, backlog 18.24 pkt
RUN 35	fq_pie_staged_s2	fq_pie, staged workload, seed 2, goodput 9.1944 Mbps, p95 RTT 23.2 ms, backlog 18.87 pkt

ID	run	summary
RUN 36	fq_pie_staged_s3	fq_pie, staged workload, seed 3, goodput 9.2358 Mbps, p95 RTT 23.2 ms, backlog 18.71 pkt
RUN 37	fq_pie_steady_s1	fq_pie, steady workload, seed 1, goodput 9.4285 Mbps, p95 RTT 23.3 ms, backlog 11.62 pkt
RUN 38	fq_pie_steady_s2	fq_pie, steady workload, seed 2, goodput 9.4996 Mbps, p95 RTT 23.2 ms, backlog 12.73 pkt
RUN 39	fq_pie_steady_s3	fq_pie, steady workload, seed 3, goodput 9.4297 Mbps, p95 RTT 23.3 ms, backlog 11.47 pkt
RUN 40	pfifo_staged_s1	pfifo, staged workload, seed 1, goodput 8.5918 Mbps, p95 RTT 2339.0 ms, backlog 564.36 pkt
RUN 41	pfifo_staged_s2	pfifo, staged workload, seed 2, goodput 8.303 Mbps, p95 RTT 2300.0 ms, backlog 558.08 pkt
RUN 42	pfifo_staged_s3	pfifo, staged workload, seed 3, goodput 8.2458 Mbps, p95 RTT 2315.0 ms, backlog 588.19 pkt
RUN 43	pfifo_steady_s1	pfifo, steady workload, seed 1, goodput 9.4101 Mbps, p95 RTT 2359.0 ms, backlog 778.62 pkt
RUN 44	pfifo_steady_s2	pfifo, steady workload, seed 2, goodput 9.4736 Mbps, p95 RTT 2339.0 ms, backlog 793.82 pkt
RUN 45	pfifo_steady_s3	pfifo, steady workload, seed 3, goodput 9.4757 Mbps, p95 RTT 2334.0 ms, backlog 795.29 pkt
RUN 46	pie_staged_s1	pie, staged workload, seed 1, goodput 9.1959 Mbps, p95 RTT 51.0 ms, backlog 10.75 pkt
RUN 47	pie_staged_s2	pie, staged workload, seed 2, goodput 9.2418 Mbps, p95 RTT 47.8 ms, backlog 10.0 pkt
RUN 48	pie_staged_s3	pie, staged workload, seed 3, goodput 9.2322 Mbps, p95 RTT 46.9 ms, backlog 10.24 pkt
RUN 49	pie_steady_s1	pie, steady workload, seed 1, goodput 9.406 Mbps, p95 RTT 47.4 ms, backlog 8.7 pkt
RUN 50	pie_steady_s2	pie, steady workload, seed 2, goodput 9.4254 Mbps, p95 RTT 47.2 ms, backlog 8.77 pkt
RUN 51	pie_steady_s3	pie, steady workload, seed 3, goodput 9.4072 Mbps, p95 RTT 46.2 ms, backlog 9.05 pkt
RUN 52	red_staged_s1	red, staged workload, seed 1, goodput 9.2535 Mbps, p95 RTT 94.2 ms, backlog 21.39 pkt
RUN 53	red_staged_s2	red, staged workload, seed 2, goodput 9.1977 Mbps, p95 RTT 94.0 ms, backlog 22.31 pkt
RUN 54	red_staged_s3	red, staged workload, seed 3, goodput 9.2849 Mbps, p95 RTT 94.3 ms, backlog 21.73 pkt
RUN 55	red_steady_s1	red, steady workload, seed 1, goodput 9.4726 Mbps, p95 RTT 87.3 ms, backlog 26.57 pkt
RUN 56	red_steady_s2	red, steady workload, seed 2, goodput 9.3863 Mbps, p95 RTT 88.3 ms, backlog 26.63 pkt
RUN 57	red_steady_s3	red, steady workload, seed 3, goodput 9.4894 Mbps, p95 RTT 86.5 ms, backlog 26.85 pkt
RUN 58	sfq_staged_s1	sfq, staged workload, seed 1, goodput 9.1821 Mbps, p95 RTT 57.4 ms, backlog 96.93 pkt
RUN 59	sfq_staged_s2	sfq, staged workload, seed 2, goodput 9.4066 Mbps, p95 RTT 55.0 ms, backlog 92.86 pkt
RUN 60	sfq_staged_s3	sfq, staged workload, seed 3, goodput 9.1979 Mbps, p95 RTT 57.4 ms, backlog 99.79 pkt

ID	run	summary
RUN 61	<code>sfq_steady_s1</code>	sfq, steady workload, seed 1, goodput 9.3558 Mbps, p95 RTT 36.2 ms, backlog 117.58 pkt
RUN 62	<code>sfq_steady_s2</code>	sfq, steady workload, seed 2, goodput 9.3542 Mbps, p95 RTT 36.1 ms, backlog 115.49 pkt
RUN 63	<code>sfq_steady_s3</code>	sfq, steady workload, seed 3, goodput 9.4238 Mbps, p95 RTT 36.0 ms, backlog 116.42 pkt

D Reproducing every number

```
python3 src/analyse.py          # tables and fact macros
python3 src/analyse_law.py      # the scaling law
python3 src/plots.py           # all analysis figures
python3 src/make_index.py       # the figure index
python3 paper/make_report.py    # this document
```

Pre-registered hypotheses and their decision thresholds are in `docs/SCALING_LAW.md`, `docs/CROSS_AQM_PREREQ.md`, `docs/SERIALIZATION_PREREQ.md` and `docs/GATE_PREREQ.md`. The defects of Section 7 are documented in `verification/`.

References

- [1] J. Gettys and K. Nichols, “Bufferbloat: dark buffers in the Internet,” *Commun. ACM*, vol. 55, no. 1, pp. 57–65, 2012.
- [2] F. Baker and G. Fairhurst, “IETF recommendations regarding active queue management,” IETF, RFC 7567, July 2015, bCP 197.
- [3] S. Floyd and V. Jacobson, “Random early detection gateways for congestion avoidance,” *IEEE/ACM Trans. Netw.*, vol. 1, no. 4, pp. 397–413, 1993.
- [4] S. Floyd, R. Gummadi, and S. Shenker, “Adaptive RED: an algorithm for increasing the robustness of RED’s active queue management,” ICSI, Tech. Rep., 2001.
- [5] R. Pan, P. Natarajan, F. Baker, and G. White, “Proportional integral controller enhanced (PIE),” IETF, Tech. Rep. RFC 8033, 2017.
- [6] K. Nichols and V. Jacobson, “Controlling queue delay,” *ACM Queue*, vol. 10, no. 5, pp. 20–34, 2012.
- [7] K. Nichols, V. Jacobson, A. McGregor, and J. Iyengar, “Controlled delay active queue management,” IETF, Tech. Rep. RFC 8289, 2018.
- [8] T. Høiland-Jørgensen, P. McKenney, D. Täht, J. Gettys, and E. Dumazet, “The FlowQueue-CoDel packet scheduler and active queue management algorithm,” IETF, Tech. Rep. RFC 8290, 2018.
- [9] T. Høiland-Jørgensen, D. Täht, and J. Morton, “Piece of CAKE: a comprehensive queue management solution for home gateways,” in *Proc. IEEE LANMAN*, 2018.
- [10] G. Ramakrishnan, M. Bhasi, V. Saicharan, L. Monis, S. D. Patil, and M. P. Tahiliani, “FQ-PIE queue discipline in the Linux kernel: design, implementation and challenges,” in *Proc. IEEE LCN*, 2019.

- [11] B. Briscoe, K. De Schepper, M. Bagnulo, and G. White, “Low latency, low loss and scalable throughput (L4S) Internet service: architecture,” IETF, Tech. Rep. RFC 9330, 2023.
- [12] K. De Schepper, B. Briscoe, and G. White, “Dual-queue coupled active queue management for L4S,” IETF, Tech. Rep. RFC 9332, 2023.
- [13] J. Ye and K.-C. Leung, “Adaptive and stable delay control for combating bufferbloat: theory and algorithms,” *IEEE Syst. J.*, vol. 14, no. 1, pp. 1285–1296, 2020.
- [14] M. Dery, O. Krupnik, and I. Keslassy, “QueuePilot: reviving small buffers with a learned AQM policy,” in *Proc. IEEE INFOCOM*, 2023.
- [15] S. R. Pokhrel, D. Satish, J. Kua, and A. Walid, “Distilling large language models for network active queue management,” *IEEE Trans. Netw.*, 2025.
- [16] F. Rodriguez *et al.*, “DESiRED: dynamic, enhanced and smart iRED, a P4-AQM with deep reinforcement learning and in-band network telemetry,” *Comput. Netw.*, vol. 244, 2024.
- [17] M. P. Toopchinezhad and M. Ahmadi, “Machine learning approaches for active queue management: a survey, taxonomy and future directions,” *Comput. Netw.*, vol. 262, 2025.
- [18] E. Sharafzadeh, R. Matson, J. Tourrilhes, P. Sharma, and S. Ghorbani, “Self-clocked round-robin packet scheduling,” in *Proc. USENIX NSDI*, 2025.
- [19] S. Ray, T. Sharma, J. Marques, P. Schmitt, F. Bronzino, and N. Feamster, “Characterizing the impact of active queue management on speed test measurements,” *arXiv:2511.19213*, 2025.
- [20] C. Hung and P. Wang, “eBPF qdisc: a generic building block for traffic control,” in *Netdev 0x17*, 2023.
- [21] V. S. Borkar, “Stochastic approximation with two time scales,” *Syst. Control Lett.*, vol. 29, no. 5, pp. 291–294, 1997.
- [22] S. Ha, I. Rhee, and L. Xu, “CUBIC: a new TCP-friendly high-speed TCP variant,” *ACM SIGOPS Oper. Syst. Rev.*, vol. 42, no. 5, pp. 64–74, 2008.
- [23] R. Jain, D.-M. Chiu, and W. Hawe, “A quantitative measure of fairness and discrimination for resource allocation in shared computer systems,” DEC, Tech. Rep. TR-301, 1984.